

Analisi di Manutenzione

Vaporant

Riferimento	VAPORANT_AM
Versione	2.0
Data	17/07/2026
Destinatario	Prof.re Andrea De Lucia
Presentato da	- Gianfranco Barba - Francesco Corcione
Approvato da	//

Revision History

Versione	Data	Descrizione	Autori
1.0	10/06/2026	Prima emissione: struttura del documento + impact analysis CR_01 (SIS/CIS/DIS/FPIS/AIS + metriche IR).	GB-FC
1.1	10/06/2026	Formalizzate le metriche (EstError/Adequacy/Sharpness/Effectiveness) e la scala d'impatto; aggiunte Fattibilità e Rischi.	GB
1.2	11/06/2026	Impact analysis CR_02 (SIS/CIS/DIS/FPIS/AIS, verifica, metriche).	GB
1.3	12/06/2026	Regressione conclusiva CR_02.	GB
1.4	12/06/2026	Uniformato lo stile di CR_02 a quello di CR_01.	GB-FC
1.5	15/06/2026	Impact analysis CR_03 (SIS/CIS/DIS/FPIS/AIS + metriche)	FC
1.6	16/06/2026	Impact analysis a priori CR_04 (identificazione, rischi, nuovo RF_GC_26, SIS).	FC
1.7	03/07/2026	Completato il documento per tutte le CR. Sezione 'Sistema modificato' riscritta.	FC
1.8	04/07/2026	Revisione e correzione degli errori.	GB-FC
1.9	14/07/2026	Riferimenti ai diagrammi UML/EER.	GB
2.0	17/07/2026	Revisione finale e consegna.	GB-FC

1 Sommario

Revision History	2
1. Scopo del documento	5
2. Panoramica del sistema attuale.....	5
2.1 Reverse engineering.....	5
2.2 Descrizione del sistema	5
2.3 Attori del sistema	5
2.4 Requisiti funzionali	6
2.5 Architettura del sistema.....	9
2.6 Database	10
2.6.1 Modello EER.....	10
2.7 Testing.....	10
3. Attività di manutenzione	10
3.1 CR_01	11
3.1.1 Identificazione, analisi e valutazione	11
3.1.2 Impact analysis.....	13
3.1.3 Risultati dell'impact analysis	15
3.1.3.3 Identificazione dell'Actual Impact Set (AIS)	16
3.1.1 3.1.4 Metriche.....	18
3.2 CR_02	19
3.2.1 Identificazione, analisi e valutazione	19
3.2.2 Impact analysis.....	21
3.2.3 Risultati dell'impact analysis	26
3.2.4 Stato di verifica (regressione).....	32
3.2.5 Ristrutturazione del repository (intervento di chiusura).....	33
3.2.6 Metriche.....	34
3.3 CR_03	34
3.3.1 Identificazione, analisi e valutazione	34
3.3.2 Impact analysis.....	36
3.3.3 Risultati dell'impact analysis	37
3.3.4 Stato di verifica (regressione).....	39
3.3.5 Metriche.....	39
3.4 CR_04	40
3.4.2 Impact analysis.....	42
3.4.3 Risultati dell'impact analysis	43
3.4.4 Stato di verifica (regressione).....	44
3.4.5 Metriche.....	45

3.5 CR_05	45
3.5.1 Identificazione, analisi e valutazione	45
3.5.2 Impact analysis a priori (SIS → CIS).....	46
3.5.3 Risultati dell'impact analysis	46
3.5.4 Stato di verifica (portabilità)	47
3.5.5 Metriche.....	48
3.6 Metriche: sintesi complessiva	48
4. Sistema modificato	49
4.1 Descrizione del sistema	49
4.2 Attori del sistema	50
4.3 Requisiti funzionali	50
4.4 Architettura del sistema.....	53
4.5 Database	55
4.5.1 Modello EER.....	55
4.6 Stato di verifica finale	55

1. Scopo del documento

Lo scopo del documento è illustrare l'attività di manutenzione svolta sul software Vaporant. Per ogni Change Request viene condotta l'impact analysis: si identificano le componenti impattate, se ne valuta il grado d'impatto e, dopo l'implementazione, si misura con metriche di Information Retrieval quanto la previsione si è rivelata accurata.

2. Panoramica del sistema attuale

2.1 Reverse engineering

Vaporant è stato consegnato privo di una documentazione esaustiva e corretta; è stata quindi condotta un'attività di reverse engineering finalizzata alla ricostruzione dei requisiti funzionali. L'attività ha seguito il paradigma goals/models/tools:

- **Goals:** ricostruire una documentazione di alto livello: i requisiti funzionali e le relazioni fra gli elementi del codice.
- **Models:** due modelli prodotti: la lista dei requisiti funzionali (con descrizione) e la matrice di tracciabilità, per mantenere la tracciabilità *forward* e *backward*.
- **Tools:** le funzionalità find usages e find in files di IntelliJ (analisi lessicale e sintattica per individuare utilizzi e riferimenti degli elementi del codice).

2.2 Descrizione del sistema

Vaporant è una piattaforma di *e-commerce* per la vendita di sigarette elettroniche e accessori da svapo. Offre: consultazione del catalogo prodotti (con dettaglio e ricerca), gestione del carrello (disponibile fin dalla navigazione da ospite), ordine con checkout e generazione della fattura in PDF, e un'area amministrativa per la gestione dei prodotti (inserimento e rimozione a catalogo). La disponibilità a magazzino è aggiornata automaticamente a seguito degli acquisti e vincola le quantità inseribili nel carrello.

2.3 Attori del sistema

- **Ospite:** utente non autenticato: consulta catalogo e dettaglio, ricerca prodotti, gestisce il carrello, si registra, consulta le pagine informative (l'acquisto richiede però registrazione e login).
- **Cliente:** utente registrato e autenticato: eredita le funzionalità dell'ospite e in più gestisce profilo e indirizzi, effettua l'acquisto, consulta lo storico ordini e scarica la fattura.
- **Amministratore:** gestisce il catalogo: login/logout e profilo come il cliente, accesso alla vista amministrativa, inserimento e rimozione di prodotti.

2.4 Requisiti funzionali

Sono stati ricostruiti 25 requisiti funzionali (identificativo RF_<sigla>_<n>, priorità Alta/Media/Bassa), organizzati in 8 sottosistemi:

Sottosistema	Sigla	RF
Gestione Autenticazione	GA	RF_GA_1-2
Gestione Registrazione	GR	RF_GR_3
Gestione Profilo	GP	RF_GP_4-10
Gestione Catalogo	GC	RF_GC_11-14
Gestione Carrello	GCR	RF_GCR_15-18
Gestione Ordine	GO	RF_GO_19-22
Gestione Prodotti	GPR	RF_GPR_23-24
Gestione Generale	GG	RF_GG_25

Elenco completo dei requisiti funzionali:

RF	Sottosistema	Enunciato
RF_GA_1	Autenticazione	Il sistema consente il login dell'utente registrato.
RF_GA_2	Autenticazione	Il sistema consente il logout dell'utente autenticato.
RF_GR_3	Registrazione	Il sistema consente la registrazione di un nuovo cliente.
RF_GP_4	Profilo	Il sistema consente la visualizzazione del profilo utente.

RF_GP_5	Profilo	Il sistema consente la modifica dell'e-mail dell'utente.
RF_GP_6	Profilo	Il sistema consente la modifica del numero di telefono dell'utente.
RF_GP_7	Profilo	Il sistema consente la modifica della password dell'utente.
RF_GP_8	Profilo	Il sistema consente la visualizzazione degli indirizzi salvati dall'utente.
RF_GP_9	Profilo	Il sistema consente l'aggiunta di un nuovo indirizzo.
RF_GP_10	Profilo	Il sistema consente la visualizzazione dello storico degli ordini effettuati.
RF_GC_11	Catalogo	Il sistema consente la visualizzazione del catalogo prodotti.
RF_GC_12	Catalogo	Il sistema consente l'ordinamento del catalogo prodotti.
RF_GC_13	Catalogo	Il sistema consente la visualizzazione del dettaglio di un prodotto.
RF_GC_14	Catalogo	Il sistema consente la ricerca di un prodotto nel catalogo.

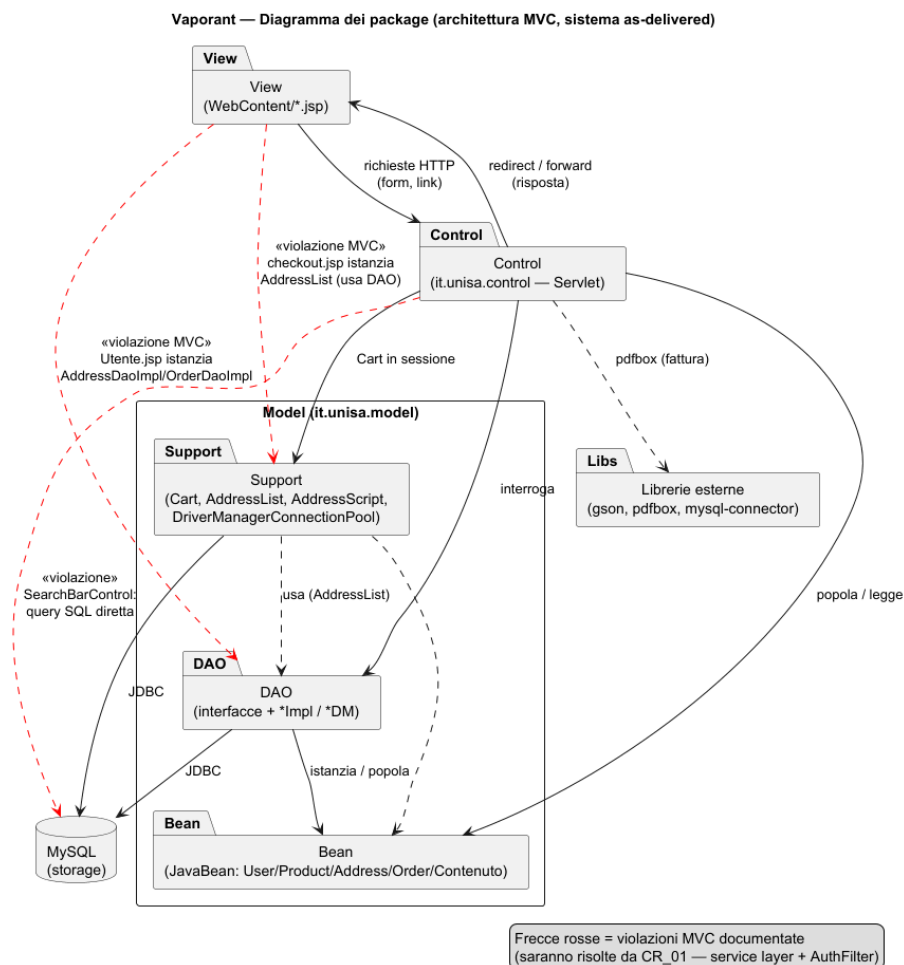
RF_GCR_15	Carrello	Il sistema consente la visualizzazione del carrello.
RF_GCR_16	Carrello	Il sistema consente l'aggiunta di un prodotto al carrello.
RF_GCR_17	Carrello	Il sistema consente la rimozione di un prodotto dal carrello.
RF_GCR_18	Carrello	Il sistema consente l'aggiornamento della quantità di un prodotto nel carrello.
RF_GO_19	Ordine	Il sistema consente la visualizzazione del checkout.
RF_GO_20	Ordine	Il sistema consente di effettuare l'acquisto.
RF_GO_21	Ordine	Il sistema consente la visualizzazione del riepilogo dell'ordine effettuato.
RF_GO_22	Ordine	Il sistema consente lo scaricamento della fattura relativa a un ordine.
RF_GPR_23	Prodotti	Il sistema consente all'amministratore di inserire un nuovo prodotto nel catalogo.
RF_GPR_24	Prodotti	Il sistema consente all'amministratore di rimuovere un prodotto dal catalogo.

2.5 Architettura del sistema

L'architettura adottata è la Model-View-Controller (MVC):

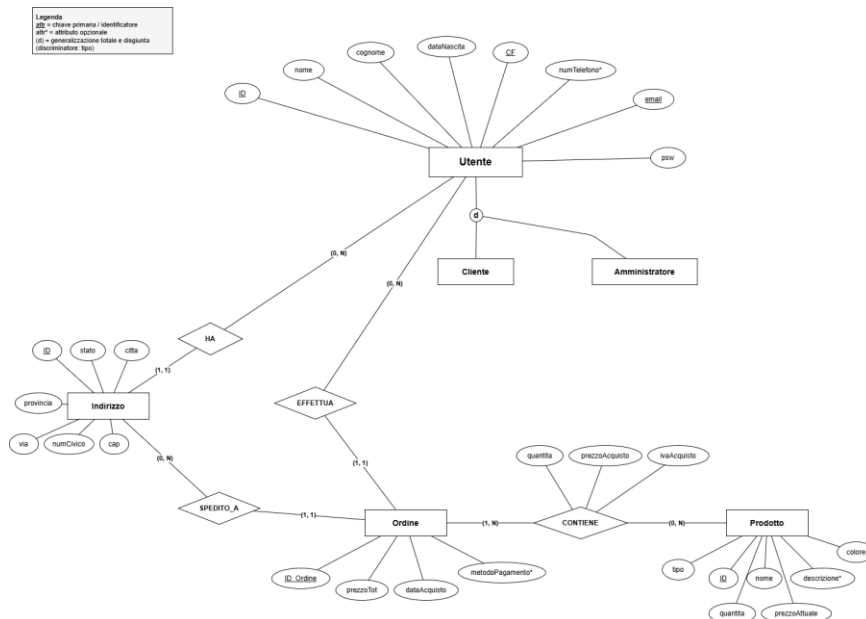
- **Model** — JavaBean (UserBean, ProductBean, AddressBean, OrderBean, ContenutoBean) e DAO (UserDaoImpl, ProductModelDM, AddressDaoImpl, OrderDaoImpl, ContenutoDaoImpl), in src/it/unisa/model/.
- **View** — le JSP (ProductView.jsp, DetailsView.jsp, CartView.jsp, checkout.jsp, Utente.jsp, ...), in WebContent/.
- **Controller** — le Servlet (LoginControl, ProductControl, CartControl, OrderControl, ModifyControl, ...), in src/it/unisa/control/, mappate sugli URL nel web.xml.

L'accesso al database MySQL storage avviene via JDBC, attraverso un *data source* JNDI (jdbc/storage).



2.6 Database

2.6.1 Modello EER



L'analisi ha rilevato criticità di schema, fra cui l'incoerenza chiave Utente.indirizzoFatt, presente nel codice ma assente nello script SQL, corretta in CR_01.

2.7 Testing

L'as-delivered è privo di qualsiasi attività di testing (nessun test automatico, nessun framework, nessuna suite di regressione o copertura). Per questo, prima di ogni modifica, è stato condotto un testing di sistema black-box con oracoli *requirement-based*, così da fotografare il comportamento as-delivered e costituire la baseline che abilita il testing di regressione a valle della manutenzione.

3. Attività di manutenzione

Il presente paragrafo illustra le attività di manutenzione svolte. Ciascun sotto-paragrafo riporta una Change Request e, per essa, conduce l'**impact analysis** e ne riporta i **risultati**. Le Change Request sono affrontate **sequenzialmente** secondo l'ordine d'esecuzione del progetto: al termine dell'analisi e dell'implementazione di una, si procede con la successiva.

Modalità di identificazione delle componenti. Per ogni Change Request l'identificazione delle componenti impattate si è basata sulla **matrice di tracciabilità** (mappatura RF ↔ classi ↔ test case) e, a supporto, su una ricerca nel codice condotta con tecniche di **Information Retrieval** integrate in IntelliJ, *find in files* e *find usages* (analisi lessicale e sintattica per individuare utilizzi e riferimenti di elementi del codice, ad esempio i metodi).

L'impact analysis ha portato a definire, per ogni Change Request:

- **Start Impact Set (SIS)**, l'insieme iniziale delle componenti che si presume verranno impattate dalla modifica;
- **Candidate Impact Set (CIS)**, l'insieme stimato delle componenti impattate (include il SIS più le componenti raggiunte per *ripple effect*).

Per ogni artefatto è indicato il **grado d'impatto**. L'intensità dell'impatto non è un'etichetta arbitraria: è determinata dal **ripple effect** (effetto a catena), ossia da come una modifica si **propaga** dal componente toccato verso gli altri (*change propagation*). La scala è quindi fondata sulla natura della propagazione:

- **Forte**, modifiche **strutturali alle interfacce** del componente (firme di metodi, schema DB, contratto pubblico) che inducono un elevato *ripple effect* **inter-modulo** (la modifica si propaga ai moduli dipendenti); rientra qui anche la creazione/sostituzione integrale di un artefatto;
- **Medio**, modifiche alla **logica interna** (es. query SQL, flussi interni) con *change propagation* **intra-modulo**, **senza** alterare le interfacce verso l'esterno;
- **Debole**, modifiche **marginali** confinate a singole stringhe/commenti (o **dipendenza in sola lettura**): impatto quasi nullo sulla complessità, nessuna propagazione.

A valle dell'implementazione si riportano i **risultati dell'impact analysis**:

- **Discovered Impact Set (DIS)**, le componenti scoperte *durante* l'implementazione (non previste nel CIS);
- **False Positive Impact Set (FPIS)**, le componenti presenti nel CIS che però **non** sono state realmente modificate;
- **Actual Impact Set (AIS)**, l'insieme di **tutte** le componenti realmente modificate, pari a (CIS – FPIS) U DIS.

3.1 CR_01

3.1.1 Identificazione, analisi e valutazione

Campo	Valore
Identificativo	CR_01
Titolo	Correzione dei difetti
Tipologia	Correttiva
Priorità	Alta

Dipendenze	Nessuna (è la prima Change Request, propedeutica)
Origine	Difetti rilevati in fase di reverse engineering e nel testing di baseline
Motivazione	Il sistema as-delivered presenta difetti bloccanti (incoerenza schema↔codice sulla colonna Utente.indirizzoFatt, campi tipo/colore prodotto NOT NULL non valorizzati, CHECK non portabile) che impediscono login, registrazione, checkout e inserimento prodotto; oltre a difetti applicativi che alterano il comportamento atteso dai requisiti (controllo ruolo admin per riferimento, doppio scalo del magazzino, race sul recupero dell'ID ordine, valuta errata, logout ridondante, sintassi DELETE * FROM, fattura non generabile, ecc.).
Obiettivo	Rendere il sistema funzionalmente corretto e produrre una baseline di test verde, prerequisito per il re-engineering (CR_02).
Analisi di Fattibilità (Feasibility)	L'intervento è fattibile: si limita a correggere DAO, controlli JSP e schema del database. Soluzioni alternative di portata maggiore (normalizzazione dell'indirizzo via relazione Indirizzo, rilocalizzazione della logica di magazzino in un service layer) sono state valutate e rimandate a CR_02 per contenere il rischio.
Analisi dei Rischi (Risk)	Rischio principale: introdurre nuovi difetti (regressioni) nel codice legacy privo di test. Mitigazione: la suite di test di baseline è pronta per essere rieseguita come regressione test-all a valle delle modifiche. Si è adottata la prassi di un commit per ogni difetto corretto, per poter isolare e annullare singolarmente ogni intervento. Nessuna implicazione di sicurezza introdotta: gli interventi di security sono isolati in CR_03.

Elenco degli interventi nei dettagli:

- Allineamento dello schema del database al codice esistente: introduzione della colonna dell'indirizzo di fatturazione mancante nella tabella Utente, così da sbloccare le funzionalità di login, registrazione e checkout che già la richiedevano lato applicativo.
- Correzione della portabilità di un vincolo di integrità sul metodo di pagamento, riscritto con sintassi SQL standard anziché una sintassi specifica non portabile.
- Completamento della modellazione del prodotto: introduzione delle proprietà "tipo" e "colore" (obbligatorie a schema ma finora non gestite), con relativo adeguamento del form di inserimento e della logica di salvataggio, incluso il trattamento corretto del prezzo come valore decimale entro i limiti ammessi.
- Correzione del controllo di accesso all'area amministrativa, oggi basato su un confronto di stringhe non affidabile e privo di un'interruzione del flusso dopo il rifiuto d'accesso.

- Eliminazione del doppio decremento delle scorte di magazzino in fase di acquisto, che oggi avviene due volte con parametri incoerenti.
- Correzione della sintassi non valida delle query di cancellazione nei componenti di accesso ai dati di utenti, contenuti d'ordine e ordini.
- Riscrittura della generazione della fattura: sostituzione di un riferimento a un percorso di file assoluto e a un modello inesistente con una generazione del documento interamente programmatica, incluso il ripristino del simbolo di valuta corretto.
- Correzione del simbolo di valuta mostrato nel totale del carrello in fase di checkout.
- Semplificazione della procedura di logout, oggi affetta da una gestione ridondante e potenzialmente insicura della sessione.
- Rimozione di un'istruzione duplicata nell'impostazione dell'identificativo degli indirizzi, presente in più metodi di accesso ai dati.
- Sostituzione di riferimenti a URL assoluti nelle pagine di errore con percorsi relativi all'applicazione.
- Adozione di un tipo di dato a precisione esatta per gli importi monetari relativi al prezzo d'acquisto, in luogo di un tipo a virgola mobile.
- Rimozione di istruzioni di stampa residue lasciate a scopo di debug in fase di sviluppo.
- Correzione di un difetto di correttezza nel recupero dell'identificativo di un ordine appena creato, che in condizioni di concorrenza poteva associare i contenuti dell'ordine a un ordine errato; l'identificativo viene ora recuperato in modo affidabile dalla stessa operazione di inserimento.
- Rimozione di un'istruzione che azzerava impropriamente il contatore degli identificativi prodotto ad ogni cancellazione, con rischio di corruzione della sequenza.
- Creazione della pagina di errore mancante richiamata dal flusso di generazione fattura.

3.1.2 Impact analysis

3.1.2.1 Identificazione dello Start Impact Set (SIS)

Artefatti **direttamente** interessati dagli interventi precedentemente elencati.

#	Artefatto	Descrizione dell'intervento	Grado
1	db/storage.sql	Aggiunta colonna indirizzoFatt a Utente (nullable); CHECK metodoPagamento con apici singoli (portabilità)	Forte
2	ProductBean.java	Aggiunta degli attributi tipo e colore (campi, costruttore, getter/setter)	Forte

3	ProductViewAdmin.jsp	Form: input tipo/colore e prezzo min=0.1; correzione controllo ruolo admin (confronto per valore, return, null-guard)	Forte
4	ProductControl.java	Lettura dei parametri tipo/colore; prezzo come decimale	Medio
5	ProductModelDM.java	doSave: INSERT completo (tipo/colore/prezzo); doDelete: rimozione del reset auto_increment	Medio
6	UserDaoImpl.java	Correzione sintassi DELETE * FROM → DELETE FROM	Debole
7	ContenutoDaoImpl.java	Correzione DELETE; rimozione del doppio scalo magazzino; prezzoAcquisto → BigDecimal	Medio
8	OrderDaoImpl.java	Correzione DELETE; saveOrder con <i>generated keys</i> ; rimozione di getIdfromDB (race)	Forte
9	AddressDaoImpl.java	Rimozione della doppia setId (3 metodi); rimozione statement di debug	Debole
10	ContenutoBean.java	prezzoAcquisto da float a BigDecimal (precisione monetaria)	Medio
11	OrderControl.java	Uso dell'ID ordine generato; ContenutoBean con BigDecimal; rimozione debug	Medio
12	FatturaControl.java	Riscrittura: PDF generato via codice (PDFBox), senza percorso assoluto/template; simbolo valuta; guardie sui dati nulli	Forte
13	LogoutControl.java	Logout con sola invalidazione della sessione	Medio
14	checkout.jsp	Simbolo valuta £ → € nel totale	Debole
15	ordine.jsp	Rimozione degli statement di debug	Debole
16	Utente.jsp	Correzione encoding dei caratteri accentati (è, à)	Debole
17	WEB-INF/404.jsp	URL assoluto → relativo al contesto	Debole

18	WEB-INF/500.jsp	URL assoluto → relativo al contesto	Debole
19	error-page.jsp (<i>nuovo</i>)	Creazione della pagina d'errore per il flusso fattura	Forte

|SIS| = 19.

3.1.2.2 Identificazione del Candidate Impact Set (CIS)

Il CIS comprende il SIS più le componenti raggiunte per **ripple effect** (dipendenze dei precedenti). Di seguito le sole componenti **aggiuntive** rispetto al SIS.

#	Artefatto (oltre al SIS)	Perché impattato (ripple)	Grado
20	LoginControl.java	La correzione di indirizzoFatt (via UserDaoImpl.findByCred) sblocca il login	Forte
21	SignControl.java	La correzione di indirizzoFatt (via saveUser) sblocca la registrazione	Forte
22	ProductView.jsp	Visualizza ProductBean modificato (dipendenza in lettura)	Debole
23	DetailsView.jsp	Visualizza ProductBean modificato (dipendenza in lettura)	Debole
24	Cart.java	Contiene/gestisce ProductBean modificato	Debole
25	AddressBean.java	toStringScript usato nel flusso fattura riscritto (B4)	Medio
26	Header.jsp	Link login/logout nel ciclo di sessione dopo la modifica del logout (B6)	Debole

|CIS| = 26 (19 del SIS + 7 di ripple)

3.1.3 Risultati dell'impact analysis

3.1.3.1 Identificazione del Discovered Impact Set (DIS)

Componenti **non previste** nel CIS, emerse durante l'implementazione.

#	Artefatto	Perché scoperto	Grado
1	OrderDAO.java (<i>interfaccia</i>)	L'eliminazione della race su getIdfromDB ha richiesto di modificare la firma di saveOrder (ritorno dell'ID)	Medio

generato): l'adeguamento dell'interfaccia non era stato previsto nel CIS, che aveva considerato la sola implementazione OrderDaoImpl.java

|DIS| = 1.

3.1.3.2 Identificazione del False Positive Impact Set (FPIS)

Componenti presenti nel CIS che **non** sono state realmente modificate.

#	Artefatto (nel CIS)	Perché non modificato
1	Utente.jsp	L'intervento di encoding previsto si è rivelato non necessario : i byte erano corretti in ISO-8859-1 (0xE0='à'), serviti come Latin-1 → render corretto; il  era un artefatto di lettura UTF-8. Riclassificato a CR_02 (uniformazione encoding a UTF-8).
2	LoginControl.java	Il login è stato sbloccato dalla correzione a monte (schema DB + UserDaoImpl): il controller non ha richiesto modifiche.
3	SignControl.java	La registrazione è stata sbloccata a monte (schema DB + saveUser): il controller non ha richiesto modifiche.
4	ProductView.jsp	Dipendenza in sola lettura : i nuovi campi tipo/colore non hanno richiesto modifiche al rendering esistente.
5	DetailView.jsp	Dipendenza in sola lettura, nessuna modifica necessaria.
6	Cart.java	Gestisce ProductBean ma non ha richiesto modifiche (i nuovi campi non incidono sulla logica del carrello).
7	AddressBean.java	La riscrittura della fattura non ha richiesto modifiche al bean (uso invariato di toStringScript).
8	Header.jsp	La semplificazione del logout non ha richiesto modifiche ai link di sessione.

|FPIS| = 8. Tutti *over-estimation* di natura conservativa: dipendenze di lettura e componenti sbloccate da correzioni a monte, più un intervento (Utente.jsp) correttamente riclassificato ad altra Change Request.

3.1.3.3 Identificazione dell'Actual Impact Set (AIS)

Componenti **realmente** modificate = (CIS – FPIS) ∪ DIS.

#	Artefatto	Modifica effettiva	Intervento di riferimento
1	db/storage.sql	Colonna indirizzoFatt su Utente + 4 INSERT seed; CHECK metodoPagamento con apici singoli	Colonna indirizzo di fatturazione; portabilità vincolo pagamento
2	ProductBean.java	Attributi tipo/colore	Modellazione tipo/colore prodotto
3	ProductControl.java	Lettura tipo/colore + prezzo decimale	Modellazione tipo/colore prodotto
4	ProductModelDM.java	INSERT completo in doSave; rimozione reset auto_increment in doDelete	Modellazione tipo/colore prodotto; reset auto_increment
5	ProductViewAdmin.jsp	Input tipo/colore + prezzo; controllo ruolo admin per valore + return + null-guard	Modellazione tipo/colore prodotto; controllo ruolo admin
6	UserDaoImpl.java	DELETE * FROM → DELETE FROM	Sintassi query di cancellazione
7	ContenutoDaoImpl.java	DELETE; rimozione doppio scalo magazzino; prezzoAcquisto → BigDecimal	Sintassi cancellazione; doppio scalo magazzino; precisione monetaria
8	OrderDaoImpl.java	DELETE; saveOrder con generated keys, rimozione getIdfromDB	Sintassi cancellazione; race ID ordine
9	OrderDAO.java (<i>non previsto a priori</i>)	Adeguamento firma di saveOrder	Race ID ordine (effetto indiretto)
10	AddressDaoImpl.java	Rimozione doppia setId ×3; rimozione debug	setId duplicato; debug residuo
11	ContenutoBean.java	prezzoAcquisto float → BigDecimal	Precisione monetaria

12	OrderControl.java	Uso ID generato; BigDecimal; rimozione debug	Race ID ordine; precisione monetaria; debug residuo
13	checkout.jsp	Simbolo valuta £ → €	Simbolo di valuta
14	LogoutControl.java	Sola invalidazione sessione	Logout
15	WEB-INF/404.jsp	URL relativo al contesto	URL pagine di errore
16	WEB-INF/500.jsp	URL relativo al contesto	URL pagine di errore
17	ordine.jsp	Rimozione statement di debug	Debug residuo
18	FatturaControl.java	Generazione PDF via codice; guardie dati nulli	Fattura via codice
19	error-page.jsp (<i>nuovo</i>)	Pagina d'errore del flusso fattura	Fattura via codice

|AIS| = 19.

1.1.1 3.1.4 Metriche

Applicando le metriche di Information Retrieval:

- $\text{Recall} = |\text{CIS} \cap \text{AIS}| / |\text{AIS}|$;
- $\text{Precision} = |\text{CIS} \cap \text{AIS}| / |\text{CIS}|$;
- F-measure = media armonica di Precision e Recall;
- Errore di stima dell'impatto $\text{EstError} = (|\text{DIS}| + |\text{FPIS}|) / |\text{CIS}|$ ai valori di CR_01
- ($|\text{CIS}| = 26$, $|\text{AIS}| = 19$, $|\text{CIS} \cap \text{AIS}| = 18$, $|\text{DIS}| = 1$, $|\text{FPIS}| = 8$):
 - **Recall = 0.95, Precision = 0.69, F-measure = 0.80, EstError = 0.35.**

Il Recall elevato indica che l'analisi a priori ha catturato quasi tutte le componenti modificate: l'unico impatto mancato è l'interfaccia del DAO dell'ordine, conseguenza della modifica della firma del salvataggio. La Precision più contenuta riflette una sovrastima conservativa del ripple (dipendenze in sola lettura, componenti sbloccate da correzioni a monte, un intervento riclassificato ad altra Change Request) — bilanciamento desiderabile per una manutenzione correttiva propedeutica, che privilegia la copertura tollerando falsi positivi a basso costo. Con un solo impatto mancato (DIS non vuoto), l'analisi non è pienamente adeguata, ma l'inadeguatezza è minima e l'errore è dominato da falsi positivi a basso costo.

3.2 CR_02

3.2.1 Identificazione, analisi e valutazione

Campo	Valore
Identificativo	CR_02
Titolo	Re-engineering architetturale
Tipologia	Perfettiva
Priorità	Alta
Dipendenze	CR_01 (baseline funzionalmente corretta + rete di regressione)
Origine	Assenza di un service layer (logica applicativa dispersa in Servlet/JSP) e assenza di una strategia di gestione della sessione; incident emersi dalla regressione di CR_01 (connessioni/transazioni incoerenti, area riservata non protetta, DAO non uniformi, validazione input assente, disallineamento sessione↔DB)
Motivazione	Il sistema, pur corretto dopo CR_01, viola sistematicamente il pattern MVC: la logica di business è dispersa tra Servlet e JSP anziché in un livello dedicato; la gestione delle connessioni è eterogenea e difettosa; non esiste alcuna strategia di sessione (ogni pagina ri-interroga il DB, gli update di profilo non sono propagati); l'autenticazione sulle aree riservate è assente o incoerente; i DAO hanno naming e contratti non uniformi; manca la validazione lato server; l'indirizzo di fatturazione è un campo denormalizzato anziché una relazione
Obiettivo	Introdurre un service layer come unica sede della logica applicativa; consolidare le connessioni su un'unica strategia (DataSource) con transazioni esplicite governate dal service; introdurre una strategia di sessione e un filtro di autenticazione centralizzato; uniformare il layer DAO; aggiungere la validazione input lato server; normalizzare l'indirizzo di fatturazione, preservando il comportamento sui flussi già verdi e correggendo (flip Failed→Passed) gli incident con oracolo già requirement-based.
Analisi di Fattibilità (Feasibility)	Intervento di portata architetturale, reso fattibile da: la baseline verde e la rete di regressione prodotte da CR_01 (refactor sotto rete di sicurezza); decisioni architetturali fissate a priori (DataSource unico, confini transazionali nel service, logging senza nuove dipendenze, normalizzazione indirizzo) che eliminano l'ambiguità di design; un ordine a fasi con dipendenze esplicite, che applica

	prima gli interventi a comportamento preservato e poi quelli correttivi; lavorazione per unità coerenti con checkpoint di regressione dopo ogni intervento
Analisi dei Rischi (Risk)	Rischio principale: effetto a catena indesiderato sui flussi oggi verdi durante un refactor esteso. Mitigazione: verifica analitica di tutti i chiamanti prima di ogni checkpoint, poi esecuzione della suite. Rischio specifico sullo schema: la normalizzazione dell'indirizzo introduce una dipendenza tra Utente e Indirizzo, mitigata con ALTER TABLE post-creazione (nessuna migrazione dati richiesta, seed già nullo).
Effetto atteso sul testing	Natura duale: sui casi oggi <i>Passed</i> (flussi guest, login, checkout/acquisto e fattura validi, ecc.) si attende invarianza; su un sottoinsieme di casi oggi <i>Failed</i> si attende il flip a <i>Passed</i> a oracolo invariato. I risultati confluiscono nel registro di regressione e nei report finali di esecuzione e di incident

Elenco degli interventi nei dettagli:

- Introduzione di un livello di servizio dedicato (nuovo package applicativo), unica sede della logica di business oggi dispersa tra Servlet e JSP, con conseguente riduzione dei controller al solo compito di ricevere la richiesta, invocare il servizio e smistare la risposta.
- Rilocalizzazione della logica di gestione del magazzino all'interno del nuovo livello di servizio dedicato all'ordine, in sostituzione della logica oggi presente nel controller e di un metodo ormai morto nel DAO dei contenuti d'ordine.
- Centralizzazione del calcolo dell'IVA in un unico punto del servizio, oggi invece duplicato in due componenti distinti con lo stesso valore.
- Introduzione dell'atomicità transazionale sul checkout: le operazioni di salvataggio dell'ordine, dei suoi contenuti e di aggiornamento delle scorte vengono ricondotte a un'unica transazione governata dal servizio, con commit/rollback espliciti.
- Consolidamento della strategia di connessione al database su un'unica modalità (sorgente dati unica), con eliminazione di un meccanismo di pool di connessioni alternativo oggi coesistente e causa di comportamenti incoerenti tra letture e scritture concorrenti.
- Rimozione dei commit sparsi all'interno dei singoli componenti di accesso ai dati, il cui governo transazionale viene spostato interamente nel livello di servizio.
- Uniformazione del layer di accesso ai dati: allineamento della convenzione di naming dei metodi (oggi incoerente tra i vari componenti) e chiarimento dei metodi di ricerca degli indirizzi, i cui nomi non riflettono correttamente il loro comportamento.
- Uniformazione dei contratti di ritorno dei componenti di accesso ai dati: le ricerche che restituiscono collezioni tornano una lista vuota anziché un valore nullo in assenza di risultati, e la ricerca di un singolo prodotto per identificativo inesistente restituisce un valore nullo anziché un oggetto vuoto.

- Rimozione di uno stato condiviso non protetto dalla concorrenza in un controller dell'area prodotti.
- Introduzione della gestione della sessione applicativa: definizione esplicita di quali dati caricare in sessione al login e propagazione in sessione degli aggiornamenti di profilo, oggi non sincronizzati con i dati persistenti.
- Aggiornamento delle viste che oggi ri-interrogano il database o ricostruiscono i dati a runtime, affinché leggano invece dalla sessione già popolata.
- Eliminazione di due classi di supporto nate come workaround per l'assenza di gestione della sessione, non più necessarie una volta introdotta quest'ultima.
- Introduzione di un filtro di autenticazione e autorizzazione centralizzato sulle risorse riservate dell'applicazione, oggi prive di un controllo di accesso coerente.
- Introduzione della validazione degli input lato server su registrazione, profilo, indirizzo, prodotto e carrello, oggi delegata al solo lato client e quindi aggirabile.
- Correzione della gestione robusta del dettaglio prodotto in presenza di identificativo non numerico o inesistente, oggi causa di errore non gestito.
- Correzione della gestione dell'eccezione sull'ordinamento del catalogo, oggi silenziosamente inghiottita anziché segnalata.
- Normalizzazione dell'indirizzo di fatturazione: sostituzione del campo testuale denormalizzato con una relazione dedicata verso un'entità Indirizzo, con conseguente adeguamento dello schema, dei dati seed e dei flussi di checkout/sessione che lo utilizzano.
- Pulizia del modello dell'ordine: rimozione di un contatore statico non più utilizzato e non sicuro in concorrenza.
- Adeguamento dei calcoli monetari del carrello a un tipo a precisione esatta e correzione del confronto tra prodotti, oggi basato su una rappresentazione testuale anziché sull'identificativo.
- Introduzione trasversale di una gerarchia di eccezioni applicative dedicate e di un meccanismo di logging strutturato, in sostituzione della stampa diretta a console e della propagazione grezza delle eccezioni tecniche, con rimozione contestuale del codice morto residuo.

3.2.2 Impact analysis

3.2.2.1 Identificazione dello Start Impact Set (SIS)

Artefatti direttamente interessati dagli interventi di re-engineering. Le righe 1 e 2 raggruppano più file di nuova creazione (rispettivamente 6 e 2); per il computo delle metriche si conta a livello di singolo artefatto, per coerenza con l'AIS.

#	Artefatto	Intervento	Grado
---	-----------	------------	-------

1	it.unisa.service.{UserService,OrderService,CartService, ProductService,InvoiceService,AddressService} (6 nuovi)	Introduzione del livello di servizio (logica applicativa, atomicità del checkout, centralizzazione dell'IVA)	Forte
2	it.unisa.exception.{DataAccessException,ServiceException} (2 nuovi)	Gerarchia di eccezioni applicative	Forte
3	it.unisa.control.AuthFilter (nuovo)	Filtro di autenticazione sulle aree riservate	Forte
4	control/OrderControl.java	Estrazione della logica nel servizio; logica di magazzino e atomicità del checkout; normalizzazione dell'indirizzo	Forte
5	control/ModifyControl.java	Estrazione nel servizio; sincronizzazione della sessione; validazione degli input	Forte
6	control/ProductControl.java	Estrazione nel servizio; rimozione dello stato statico; validazione degli input; gestione dell'errore di ordinamento	Forte
7	control/CartControl.java	Estrazione nel servizio; validazione della quantità	Forte
8	control/DetailsControl.java	Estrazione nel servizio; robustezza del dettaglio prodotto	Medio
9	control/AddressControl.java	Estrazione nel servizio; validazione degli input	Medio

10	control/SignControl.java	Estrazione nel servizio; validazione degli input	Medio
11	control/LoginControl.java	Estrazione nel servizio; caricamento dei dati in sessione al login	Forte
12	control/LogoutControl.java	Logging applicativo	Debole
13	control/FatturaControl.java	Estrazione nel servizio; centralizzazione dell'IVA	Medio
14	control/SearchBarControl.java	Estrazione nel servizio	Medio
15	model/ProductModelDM.java	Unificazione su DataSource; try-with- resources; uniformazione dei nomi; ritorno coerente su identificativo inesistente	Forte
16	model/ProductModel.java (interfaccia)	Uniformazione dei nomi dei metodi	Forte
17	model/UserDaoImpl.java	Unificazione su DataSource; try-with- resources; uniformazione dei nomi; normalizzazione dell'indirizzo	Forte
18	model/UserDAO.java (interfaccia)	Uniformazione dei nomi dei metodi	Forte
19	model/AddressDaoImpl.java	Unificazione su DataSource; try-with- resources; uniformazione e rinomina dei metodi di ricerca; ritorno a lista vuota	Forte

20	model/AddressDAO.java (interfaccia)	Uniformazione e rinomina dei metodi di ricerca	Forte
21	model/ContenutoDaoImpl.java	try-with-resources; uniformazione dei nomi; rimozione di metodo morto	Forte
22	Model/ContenutoDAO.java (interfaccia)	Uniformazione dei nomi dei metodi	Medio
23	model/OrderDaoImpl.java	try-with-resources; uniformazione dei nomi; ritorno a lista vuota	Forte
24	model/OrderDAO.java (interfaccia)	Uniformazione dei nomi dei metodi	Medio
25	model/DriverManagerConnectionPool.java (<i>ELIMINATO</i>)	Eliminazione del pool di connessioni custom	Forte
26	model/AddressList.java (<i>ELIMINATO</i>)	Rimozione della classe di supporto agli indirizzi	Forte
27	model/AddressScript.java (<i>ELIMINATO</i>)	Rimozione della classe di supporto agli indirizzi	Forte
28	model/Cart.java	Calcoli monetari in BigDecimal; confronto prodotti per identificativo; validazione della quantità	Forte
29	model/OrderBean.java	Rimozione del contatore statico	Medio
30	model/UserBean.java	Normalizzazione dell'indirizzo di fatturazione	Medio

31	model/AddressBean.java	Normalizzazione dell'indirizzo di fatturazione	Debole
32	WebContent/Utente.jsp	Lettura dei dati dalla sessione	Medio
33	WebContent/checkout.jsp	Lettura dei dati dalla sessione; normalizzazione dell'indirizzo	Medio
34	WebContent/WEB-INF/web.xml	Registrazione del filtro di autenticazione	Medio
35	db/storage.sql	Normalizzazione dell'indirizzo di fatturazione (schema + seed)	Forte

|SIS| = 41 artefatti (35 righe; le righe 1 e 2 espandono rispettivamente a 6 e 2 file).

3.2.2.2 Identificazione del Candidate Impact Set (CIS)

Il CIS comprende il SIS più le componenti raggiunte per ripple effect. Di seguito le sole componenti aggiuntive rispetto al SIS.

#	Artefatto (oltre al SIS)	Perché impattato (ripple)	Grado
36	WebContent/META-INF/context.xml	unico punto del DataSource dopo l'unificazione delle connessioni (ripple CR_05)	Debole
37	WebContent/ProductView.jsp	legge i prodotti dalla sessione	Debole
38	WebContent/ProductViewAdmin.jsp	legge i prodotti dalla sessione	Debole
39	WebContent/DetailView.jsp	legge il prodotto (robustezza del dettaglio)	Debole
40	WebContent/CartView.jsp	legge il carrello/totale (calcoli in BigDecimal)	Debole
41	WebContent/Header.jsp	collegamenti dell'area riservata dopo l'introduzione del filtro di autenticazione	Debole

42	WebContent/SignForm.jsp	eventuale feedback di validazione lato server	Debole
43	WebContent/AddressForm.jsp	eventuale feedback di validazione lato server	Debole
44	WebContent/loginForm.jsp	destinazione del reindirizzamento del filtro di autenticazione	Debole
45	WebContent/ordine.jsp	flusso dell'ordine	Debole
46	WebContent/error-page.jsp	flusso ordine/fattura	Debole
47	model/ProductBean.java	letto/scritto dai servizi riorganizzati	Debole

|CIS| = 53 (41 del SIS + 12 di ripple).

Esclusa dal perimetro: la matrice di tracciabilità. Era stata inclusa nel CIS a priori con grado Forte ("va aggiornata: RF↔nuove classi service"), ma per coerenza con CR_01 gli artefatti di verifica/documentazione non vengono conteggiati nelle metriche. La matrice non è stata aggiornata durante CR_02 e resta un lavoro residuo, senza incidere sulle metriche.

3.2.3 Risultati dell'impact analysis

A valle dell'implementazione, articolata in 15 unità di lavoro coerenti (più un intervento di chiusura sulla struttura del repository).

3.2.3.1 Identificazione del Discovered Impact Set (DIS)

|DIS| = 0. Nessuna componente modificata è risultata estranea al CIS: l'adeguatezza dell'analisi a priori è piena ($AIS \subseteq CIS$).

3.2.3.2 Identificazione del False Positive Impact Set (FPIS)

Componenti presenti nel CIS che **non** sono state realmente modificate.

#	Artefatto (nel CIS)	Perché non modificato
1	LogoutControl.java	Previsto per l'adozione del logging applicativo. Verificato: la classe non contiene istruzioni di stampa da sostituire (si limita all'invalidazione della sessione + reindirizzamento)
2	AddressBean.java	La normalizzazione dell'indirizzo ha spostato la chiave esterna sull'entità Utente, non sull'entità Indirizzo: il bean resta invariato

3	context.xml	Il DataSource era già configurato prima della manutenzione correttiva; l'unificazione ha portato i DAO sul DataSource esistente senza toccare la configurazione
4	ProductView.jsp	Dipendenza in sola lettura sui prodotti di sessione: l'intervento non ne ha alterato il contratto
5	ProductViewAdmin.jsp	Idem (sola lettura); inoltre già dotata di controllo di autenticazione e ruolo corretto dalla manutenzione correttiva; deliberatamente non toccata dal filtro di autenticazione
6	CartView.jsp	Legge il totale del carrello, ora in BigDecimal: per i valori in gioco la rappresentazione testuale è identica alla precedente; nessuna modifica al rendering
7	Header.jsp	I collegamenti condizionali erano già corretti; il filtro di autenticazione agisce a monte sulle risorse protette, non sul menu
8	SignForm.jsp	La validazione della registrazione è implementata lato server riusando il reindirizzamento già esistente verso la pagina; nessun feedback aggiuntivo richiesto
9	loginForm.jsp	Pagina di destinazione del reindirizzamento del filtro di autenticazione: già esistente e funzionante; nessuna modifica necessaria
10	ordine.jsp	Il flusso del checkout mantiene invariato il contratto di sessione verso questa pagina
11	error-page.jsp	Già gestiva l'assenza dell'ordine e conteneva già il testo usato come oracolo per l'errore di ordinamento, verificata esistente, non modificata

| FPIS | = 11. Tutte over-estimation di natura conservativa: dipendenze in sola lettura, configurazione già corretta a monte, interventi non applicabili al file specifico o assorbiti altrove nello stesso refactor.

3.2.3.3 Identificazione dell'Actual Impact Set (AIS)

Componenti realmente modificate = $(CIS - FPIS) \cup DIS = CIS - FPIS$ (poiché $DIS = \emptyset$).

#	Artefatto	Modifica effettiva	Unità
1	DataAccessException.java (<i>nuovo</i>)	Gerarchia di eccezioni applicative	Eccezioni applicative
2	ServiceException.java (<i>nuovo</i>)	Gerarchia di eccezioni applicative	Eccezioni applicative
3	Cart.java	Calcoli monetari in BigDecimal e confronto prodotti per identificativo; validazione della quantità	Precisione monetaria; validazione carrello
4	OrderBean.java	Importo totale in BigDecimal; rimozione del contatore statico	Precisione monetaria; pulizia del modello
5	ProductBean.java	Rimozione di un metodo morto	Pulizia del modello
6	OrderDaoImpl.java	Importo totale in BigDecimal; unificazione su DataSource; connessione passata per la transazione	Precisione monetaria; strategia connessioni; atomicità checkout
7	OrderControl.java	Propagazione dell'identificativo d'ordine generato; uniformazione dei nomi; ricablaggio sui servizi ordine/indirizzo	Precisione monetaria; naming DAO; livello di servizio
8	FatturaControl.java	Calcolo IVA in BigDecimal; uniformazione dei nomi; ricablaggio sui servizi indirizzo e fatturazione	Precisione monetaria; naming DAO; livello di servizio
9	ProductModelDM.java	Unificazione su DataSource; ritorno coerente su identificativo inesistente; uniformazione dei nomi; ricerca per nome; connessione passata per la transazione	Strategia connessioni; contratti di ritorno; naming DAO; livello di servizio; atomicità checkout

10	UserDaoImpl.java	Unificazione su DataSource; normalizzazione dell'indirizzo di fatturazione	Strategia connessioni; normalizzazione indirizzo
11	AddressDaoImpl.java	Unificazione su DataSource; ritorno a lista vuota	Strategia connessioni; contratti di ritorno
12	ContenutoDaoImpl.java	Unificazione su DataSource; rimozione di un metodo morto; connessione passata per la transazione	Strategia connessioni; atomicità checkout
13	DriverManagerConnectionPool.java (eliminato)	Eliminazione del pool di connessioni custom	Strategia connessioni
14	DetailsControl.java	Robustezza del dettaglio prodotto; uniformazione dei nomi	Robustezza dettaglio prodotto; naming DAO
15	DetailsView.jsp	Aggiunto il ramo "prodotto non trovato"	Robustezza dettaglio prodotto
16	CartControl.java	Guardia sul prodotto inesistente; uniformazione dei nomi; estrazione del servizio carrello	Contratti di ritorno; naming DAO; livello di servizio
17	ProductModel.java (interfaccia)	Uniformazione dei nomi; ricerca per nome; connessione passata per la transazione	Naming DAO; livello di servizio; atomicità checkout
18	AddressDAO.java (interfaccia)	Rinomina dei metodi di ricerca	Naming DAO
19	AddressList.java	Adeguamento ai nomi rinominati; successiva rimozione	Naming DAO; gestione sessione
20	Utente.jsp	Adeguamento ai nomi; lettura dalla sessione; rimozione di una guardia ridondante	Naming DAO; gestione sessione; filtro di autenticazione

21	ProductService.java (<i>nuovo</i>)	Servizio prodotti; connessione passata per la transazione; validazione del nome	Livello di servizio; atomicità checkout; validazione input
22	SearchBarControl.java	Ricablaggio sul servizio prodotti	Livello di servizio
23	CartService.java (<i>nuovo</i>)	Servizio carrello	Livello di servizio
24	UserService.java (<i>nuovo</i>)	Autenticazione/registrazione/profilo; aggiornamento dell'indirizzo di fatturazione; validazione degli input; adeguamento alla normalizzazione dell'indirizzo	Livello di servizio; validazione input; normalizzazione indirizzo
25	LoginControl.java	Ricablaggio sul servizio utente; caricamento di indirizzi e ordini in sessione al login	Livello di servizio; gestione sessione
26	SignControl.java	Ricablaggio sul servizio utente; validazione della registrazione; rimozione di un campo morto	Livello di servizio; validazione input
27	ModifyControl.java	Ricablaggio sul servizio utente; sincronizzazione della sessione; validazione di email/telefono/password	Livello di servizio; gestione sessione; validazione input
28	AddressService.java (<i>nuovo</i>)	Servizio indirizzi; validazione dell'indirizzo	Livello di servizio; validazione input
29	OrderService.java (<i>nuovo</i>)	Checkout; transazione atomica; lettura ordini per utente; adeguamento alla normalizzazione dell'indirizzo	Livello di servizio; atomicità checkout; gestione sessione; normalizzazione indirizzo
30	InvoiceService.java (<i>nuovo</i>)	Centralizzazione dell'IVA	Livello di servizio

31	AddressControl.java	Ricablaggio sul servizio indirizzi; sincronizzazione degli indirizzi in sessione; validazione dell'indirizzo	Livello di servizio; gestione sessione; validazione input
32	OrderDAO.java (<i>interfaccia</i>)	Connessione passata per la transazione	Atomicità checkout
33	ContenutoDAO.java (<i>interfaccia</i>)	Connessione passata per la transazione	Atomicità checkout
34	checkout.jsp	Lettura degli indirizzi dalla sessione; rimozione di una guardia ridondante; robustezza su carrello assente	Gestione sessione; filtro di autenticazione
35	AuthFilter.java (<i>nuovo</i>)	Filtro di autenticazione sulle aree riservate	Filtro di autenticazione
36	web.xml	Registrazione del filtro di autenticazione	Filtro di autenticazione
37	AddressForm.jsp	Rimozione di una guardia ridondante, sostituita dal filtro di autenticazione	Filtro di autenticazione
38	AddressScript.java (<i>eliminato</i>)	Rimozione della classe di supporto agli indirizzi	Gestione sessione
39	ProductControl.java	Rimozione dello stato statico; ricablaggio sul servizio prodotti; validazione del nome; gestione dell'errore di ordinamento	Naming DAO; livello di servizio; validazione input
40	db/storage.sql	Normalizzazione dell'indirizzo di fatturazione (da campo stringa a chiave esterna)	Normalizzazione indirizzo
41	UserBean.java	Normalizzazione dell'indirizzo di fatturazione	Normalizzazione indirizzo

42	UserDAO.java (<i>interfaccia</i>)	Adeguamento della firma di aggiornamento dell'indirizzo di fatturazione	Normalizzazione indirizzo
----	-------------------------------------	---	---------------------------

|AIS| = 42.

3.2.4 Stato di verifica (regressione)

La regressione conclusiva di CR_02 è stata eseguita come suite unica su base di dati ricreata (schema con l'indirizzo di fatturazione normalizzato) con fixture di ripristino del seed attiva.

Esito: 109 PASS / 4 FAIL.

Incident	Casi di test coinvolti	Esito
Isolamento/transazione	2 casi	Flip Failed→Passed confermato
Dettaglio prodotto non robusto	2 casi	Flip Failed→Passed confermato
Carrello non robusto (parte)	1 caso	Flip Failed→Passed confermato (anticipato in fase)
— (invarianza)	flusso d'acquisto	Invarianza confermata
Disallineamento sessione-DB	1 caso	Flip Failed→Passed confermato
Accesso area riservata assente	3 casi	Flip Failed→Passed confermato
Validazione input assente (campi vuoti/lunghezze)	18 casi	Flip Failed→Passed confermato
Validazione input assente (nome prodotto)	1 caso	Prodotto corretto (la validazione rifiuta il valore di soli spazi) ma oracolo di test difettoso (contava erroneamente i nodi duplicati di un componente grafico a ciclo continuo) → falso negativo da test, risolto con il ridisegno dell'oracolo

Validazione input assente (formato codice fiscale)	1 caso	Ricollocato a CR_03: la validazione introdotta copre la lunghezza del campo, non il formato — hardening preventivo fuori scope della CR perfetta
Carrello non robusto (resto)	1 caso	Flip Failed→Passed confermato
Eccezione di ordinamento inghiottita (parte non sicurezza)	1 caso	Flip Failed→Passed confermato
Invarianza generale	~80 flussi verdi + intera suite	Nessuna regressione

Esito netto. Target di flip di CR_02 = 30 (i 31 attesi a priori meno il caso del codice fiscale, ricollocato a CR_03): 29/30 confermati Passed; il 30° ha prodotto verificato corretto ma richiede il ridisegno dell'oracolo di test (30/30 dopo il ridisegno). Invarianza confermata sugli ~80 flussi verdi, nessun effetto a catena indesiderato. I 4 Failed residui sono tutti attesi e ricollocati a CR_03 (SQL injection sull'ordinamento, controllo di ruolo sulla rimozione prodotto via servlet, formato del codice fiscale) o risolti dal ridisegno dell'oracolo. Nessun fallimento spiegato.

Lavoro residuo (oltre alla regressione conclusiva): aggiornamento della matrice di tracciabilità con le nuove classi di servizio (escluso dal perimetro metriche); abilitazione dei test unitari e di integrazione sul livello di servizio, ora possibile grazie alla sua natura iniettibile.

3.2.5 Ristrutturazione del repository (intervento di chiusura)

Il progetto era organizzato con un livello di annidamento aggiuntivo, il progetto as-delivered preservato verbatim, mentre lo sviluppo avviene con il modulo alla radice del repository. Questa duplicazione strutturale è un intervento di *restructuring*, quindi di natura perfetta, coerente con CR_02, l'unica CR perfetta/di re-engineering del piano. Non è stato collocato in CR_05 (che risponde a un cambiamento d'ambiente, non a un'esigenza di manutenibilità interna) né in una CR dedicata (nessun nuovo requisito coinvolto). Eseguito come intervento di chiusura, dopo la validazione degli input e la normalizzazione dell'indirizzo e prima della regressione conclusiva, così che quest'ultima certifichi la struttura definitiva.

Intervento (a comportamento preservato): rinomina pura di centinaia di percorsi (contenuto identico, nessun delta semantico); eliminazione dei residui di configurazione del progetto originale non più validi; un numero contenuto di modifiche di contenuto reale (file di configurazione del repository, guida di setup, nome dell'applicazione nel descrittore di deploy, cosmetico, verificato privo di effetti su instradamento/contesto applicativo); aggiornamento meccanico dei riferimenti di percorso nella documentazione locale e nella configurazione dell'ambiente di sviluppo (zero impatto a runtime).

Esclusione dalle metriche IR di CR_02: un Candidate Impact Set per questo intervento coinciderebbe, per costruzione, con l'intero albero spostato: includerlo nel calcolo di Precision/Recall produrrebbe valori degeneri (predizione = realizzato, a delta nullo) e distorcerebbe le metriche dell'intervento architetturale, che misurano la qualità della previsione di impatto semantico. La ristrutturazione è pertanto documentata qualitativamente in questa sede e non contribuisce a $|SIS|$, $|CIS|$, $|AIS|$, $|DIS|$, $|FPIS|$ di CR_02. Le modifiche di contenuto reale (oltre alle rinomine pure) sono comunque riportate sopra per completezza e tracciabilità.

3.2.6 Metriche

Per CR_02 ($|CIS| = 53$, $|AIS| = 42$, $|CIS \cap AIS| = 42$, $|DIS| = 0$, $|FPIS| = 11$, la ristrutturazione del repository resta esclusa, per le ragioni motivate in sede propria):

Recall = 1.00, Precision = 0.79, F-measure = 0.88, EstError = 0.21.

Il Recall massimo indica che nessun impatto è sfuggito all'analisi a priori, nonostante CR_02 sia la CR più estesa del progetto. La Precision, superiore a quella di CR_01, indica una stima più mirata, con i falsi positivi attribuibili a dipendenze in sola lettura, configurazione già corretta a monte e interventi assorbiti altrove nello stesso refactor. Con DIS nullo l'analisi è pienamente adeguata, risultato non scontato data la portata dell'intervento, e la maggiore aderenza tra stima e realtà rispetto a CR_01 è attribuibile alle decisioni architetturali fissate a priori e all'ordine a fasi con dipendenze esplicite.

3.3 CR_03

3.3.1 Identificazione, analisi e valutazione

Campo	Valore
Identificativo	CR_03
Titolo	Messa in sicurezza del sistema
Tipologia	Preventiva
Priorità	Alta
Dipendenze	CR_01 (baseline funzionalmente corretta); innestabile in parallelo a CR_02, di cui assume lo stato re-ingegnerizzato (livello di servizio, indirizzo normalizzato)
Origine	Vulnerabilità di sicurezza latenti (nessuna failure ancora prodotta) rilevate nel reverse engineering e dalla regressione: password memorizzate in chiaro; parametro di ordinamento del catalogo concatenato nella query (SQL injection); password in chiaro nella

	rappresentazione testuale dell'utente (rischio di leak passivo nei log); controllo del ruolo amministrativo assente sulle operazioni di prodotto via servlet
Motivazione	Il sistema, pur funzionalmente corretto e re-ingegnerizzato, conserva vulnerabilità che espongono a compromissione dei dati utente e a SQL injection. Il loro trattamento preventivo riduce la superficie d'attacco prima che le criticità si traducano in incidenti, preservando il comportamento legittimo
Obiettivo	Introdurre la cifratura delle password (con ampliamento del campo che le ospita); la validazione a lista bianca del parametro di ordinamento; il mascheramento della password nella rappresentazione testuale dell'utente; il controllo del ruolo amministrativo sulle operazioni di prodotto via servlet. Il tutto a comportamento preservato per i flussi legittimi, salvo i due esiti che flippano a oracolo invariato
Analisi di Fattibilità (Feasibility)	Intervento confinato e a basso rischio sui flussi legittimi, reso fattibile da: la baseline verde e la rete di regressione già disponibili; una sola dipendenza nuova (libreria di cifratura); la collocazione della logica di cifratura nel livello di servizio, coerente con il re-engineering già svolto, che risolve in modo pulito la verifica in autenticazione e la sincronizzazione dell'hash nel cambio password
Analisi dei Rischi (Risk)	Rischio sull'autenticazione: la verifica passa dal confronto diretto della password alla verifica dell'hash; un disallineamento fra hash dei dati seed e credenziali legittime renderebbe l'area autenticata inaccessibile. Mitigazione: hash dei seed verificati per corrispondenza con le credenziali, ricreazione della base di dati come preconditione della regressione. Rischio sul cambio password: senza la sincronizzazione dell'hash in sessione, un secondo cambio consecutivo fallirebbe. Mitigazione: aggiornamento del riferimento utente in sessione con l'hash. Rischio di sovra-protezione del catalogo pubblico: il controllo di ruolo non deve precludere all'ospite la navigazione del catalogo. Mitigazione: controllo collocato nel solo ramo delle operazioni amministrative, lasciando accessibile il listing pubblico
Effetto atteso sul testing	Invarianza sui flussi legittimi (autenticazione, registrazione, cambio password, navigazione e flussi guest): la cifratura è trasparente, la verifica dell'hash sostituisce il confronto diretto a parità di comportamento osservabile. Flip Failed→Passed, a oracolo invariato, dei due casi relativi rispettivamente all'ordinamento/SQL injection e alla rimozione prodotto riservata all'amministratore via servlet. Nessun delta della specifica dei test

Analisi e opzioni d'implementazione (decisioni di scope):

- Algoritmo di cifratura: BCrypt (fattore di costo 12), tra i due algoritmi ammessi in fase di proposta. La cifratura risiede nel livello di servizio; il livello dati espone una ricerca utente per sola email (la verifica dell'hash avviene nel servizio) e un aggiornamento password che riceve il valore già cifrato.

- Mascheramento della password nella rappresentazione testuale dell'utente (la stringa non riporta più il valore).
- Validazione a lista bianca del parametro di ordinamento del catalogo (sole colonne note ammesse): la parte relativa alla gestione dell'eccezione era già stata chiusa dal re-engineering; qui resta la sola componente anti-SQL-injection.
- Controllo del ruolo amministrativo sulle operazioni di prodotto via servlet. Distinzione tra difetto manifesto e vulnerabilità latente: il controllo sulla vista amministrativa era un difetto manifesto, già corretto dalla manutenzione correttiva; la presente CR chiude il controllo a livello di servlet, aggirabile via richiesta diretta pur con la vista corretta — vulnerabilità latente, di pertinenza preventiva. Il controllo è collocato nel ramo delle azioni amministrative, preservando l'accesso pubblico al catalogo.
- Esclusioni (purezza del tipo preventivo): la politica di complessità delle password era stata inserita nel catalogo a priori, ma è stata riconosciuta come introduzione di un nuovo requisito di input — natura evolutiva/perfettiva, non preventiva (avrebbe richiesto un delta della specifica dei test) — e ricollocata a una futura manutenzione evolutiva. Il rifiuto del codice fiscale di soli spazi è una validazione di robustezza del dato, trattata come micro-correttivo separato, non come hardening preventivo; questa collocazione prevale sul rimando indicato in sede di chiusura della CR precedente, per coerenza con la natura preventiva e con le decisioni di scope qui assunte.

3.3.2 Impact analysis

3.3.2.1 Identificazione dello Start Impact Set (SIS)

Artefatti di prodotto direttamente interessati (analisi a priori; include l'intervento di complessità della password, poi escluso dall'ambito).

#	Artefatto	Descrizione dell'intervento	Grado
1	Libreria di cifratura BCrypt (<i>nuova</i>)	Introduzione della libreria di cifratura	Forte
2	db/storage.sql	Ampliamento del campo password (a 60 caratteri, per ospitare l'hash) e cifratura dei 4 dati seed	Forte
3	UserService.java	Cifratura in registrazione; verifica dell'hash in autenticazione e nel cambio password; (previsto) politica di complessità della password	Forte
4	UserDAO.java (<i>interfaccia</i>)	Adeguamento delle firme: ricerca per sola email; aggiornamento password con valore già cifrato	Forte
5	UserDaoImpl.java	Ricerca per sola email; aggiornamento password come pura operazione di persistenza	Forte

6	UserBean.java	Mascheramento della password nella rappresentazione testuale	Debole
7	ProductModelDM.java	Validazione a lista bianca del parametro di ordinamento	Medio
8	ProductControl.java	Controllo del ruolo amministrativo sulle operazioni di prodotto via servlet; irrobustimento del percorso pubblico del catalogo	Medio
9	ModifyControl.java	Rimozione della sincronizzazione della password in chiaro in sessione (ora gestita, cifrata, dal servizio)	Medio
10	SignForm.jsp	(previsto) vincolo lato client per la politica di complessità della password	Debole

|SIS| = 10.

3.3.2.2 Identificazione del Candidate Impact Set (CIS)

#	Artefatto (oltre al SIS)	Perché impattato (ripple)	Grado
11	LoginControl.java	l'autenticazione passa per la verifica dell'hash; deve restare invariata	Forte
12	SignControl.java	la registrazione invoca la cifratura nel servizio	Forte
13	ProductView.jsp	lettura del catalogo (ordinamento)	Debole
14	ProductViewAdmin.jsp	vista amministrativa / azioni amministrative	Debole
15	ProductService.java	invocato dalle operazioni di prodotto dopo il controllo di ruolo	Debole

|CIS| = 15 (10 del SIS + 5 di ripple). Esclusi dal perimetro, come per le CR precedenti: la matrice di tracciabilità e i test case.

3.3.3 Risultati dell'impact analysis

A valle dell'implementazione, sui soli artefatti di prodotto.

3.3.3.1 Identificazione del Discovered Impact Set (DIS)

|DIS| = 0. Nessun artefatto modificato è risultato estraneo al CIS (adeguatezza piena, AIS $\subseteq$ CIS). Osservazioni qualitative (non incidono su |DIS|):

- il controllo di ruolo è stato collocato fin dall'inizio nel ramo delle operazioni amministrative (non in testa al gestore della richiesta), così da preservare l'accesso pubblico al catalogo per l'utente ospite;

- sul medesimo gestore (già nel SIS) sono stati necessari due assestamenti di robustezza del percorso pubblico, a fronte di un difetto pre-esistente sul ramo principale, verificato e non introdotto dalla CR — stesso artefatto, nessun nuovo file;
- i dati seed cifrati sono stati rigenerati per garantirne la corrispondenza con le credenziali legittime, corrispondenza verificata.

3.3.3.2 Identificazione del False Positive Impact Set (FPIS)

#	Artefatto (nel CIS)	Perché non modificato
1	SignForm.jsp	Il vincolo lato client era previsto per la politica di complessità della password, riconosciuta estranea all'ambito preventivo e ricollocata a manutenzione evolutiva — intervento non realizzato, il file resta identico alla baseline
2	LoginControl.java	L'autenticazione è stata adeguata a monte, nel livello di servizio: il controller non ha richiesto modifiche
3	SignControl.java	La registrazione invoca il servizio già esistente; la cifratura è interna al servizio — controller invariato
4	ProductView.jsp	Dipendenza in sola lettura sul catalogo: nessuna modifica al rendering
5	ProductViewAdmin.jsp	Dipendenza in sola lettura; vista già dotata di controllo corretto dalla manutenzione correttiva — non toccata
6	ProductService.java	Invocato dalle operazioni di prodotto dopo il controllo di ruolo (a livello di servlet): nessuna modifica interna

|FPIS| = 6. Over-estimation di natura conservativa (dipendenze in sola lettura, adeguamenti assorbiti a monte nel livello di servizio, vista già corretta), più un intervento — la politica di complessità della password — escluso in itinere dall'ambito preventivo.

3.3.3.3 Identificazione dell'Actual Impact Set (AIS)

Componenti realmente modificate = (CIS – FPIS) ∪ DIS = CIS – FPIS (poiché DIS = ∅).

#	Artefatto	Modifica effettiva
1	Libreria di cifratura BCrypt (nuova)	Introduzione della libreria
2	db/storage.sql	Campo password ampliato (60 caratteri) + 4 seed cifrati
3	UserService.java	Cifratura in registrazione; verifica dell'hash in autenticazione; cambio password con verifica/cifratura e sincronizzazione dell'hash in sessione

4	UserDAO.java (<i>interfaccia</i>)	Ricerca utente per sola email; firma dell'aggiornamento password (valore cifrato)
5	UserDaoImpl.java	Ricerca per sola email; aggiornamento password come pura operazione di persistenza
6	UserBean.java	Mascheramento della password nella rappresentazione testuale
7	ProductModelDM.java	Validazione a lista bianca del parametro di ordinamento
8	ProductControl.java	Controllo del ruolo amministrativo sulle operazioni via servlet; irrobustimento del percorso pubblico del catalogo
9	ModifyControl.java	Rimozione della sincronizzazione della password in chiaro in sessione

|AIS| = 9. SignForm.jsp non figura: l'intervento previsto, escluso dall'ambito, lascia il file identico alla baseline.

3.3.4 Stato di verifica (regressione)

La regressione conclusiva di CR_03 è stata eseguita su base di dati ricreata (campo password ampliato e seed cifrati) con fixture di ripristino attiva. Esito: 112 PASS / 1 FAIL su 113 casi.

Esito	Casi	Risultato
Flip (Failed→Passed)	Ordinamento/SQL injection; rimozione prodotto riservata all'amministratore	Passed a oracolo invariato, confermato
Invarianza	Autenticazione, cambio password consecutivo, registrazione/profilo, flussi guest (catalogo, carrello, informativi)	Passed, nessuna regressione
Precondizione verificata	Corrispondenza degli hash dei 4 seed con le credenziali legittime	Confermata
Residuo	Codice fiscale di soli spazi	Failed — validazione di formato non in carico a CR_03, micro-correttivo separato

L'esito confluisce nel registro di regressione e nei report finali di esecuzione e di incident, con i due incident target di CR_03 portati a Chiuso/Verificato.

3.3.5 Metriche

Per CR_03 ($|CIS| = 15$, $|AIS| = 9$, $|CIS \cap AIS| = 9$, $|DIS| = 0$, $|FPIS| = 6$):

Recall = 1.00, Precision = 0.60, F-measure = 0.75, EstError = 0.40.

Il Recall massimo conferma che nessun impatto reale è sfuggito alla stima a priori. La Precision, inferiore a CR_01 e CR_02, riflette una sovrastima più marcata dovuta a due fattori a basso costo: la riduzione di ambito decisa in itinere (esclusione della politica di complessità password, poi ricollocata a manutenzione evolutiva) e un ripple conservativo in sola lettura. L'intero errore di stima è da falsi positivi, non da impatti mancati, bilanciamento appropriato per una manutenzione preventiva il cui catalogo è stato poi ristretto per preservarne la purezza tipologica

3.4 CR_04

Campo	Valore
Identificativo	CR_04
Titolo	Filtraggio e ricerca avanzata del catalogo
Tipologia	Perfettiva (evolutiva)
Priorità	Media
Dipendenze	CR_02 (livello di servizio del catalogo, su cui la nuova funzionalità si innesta)
Origine	Esigenza funzionale: il catalogo dispone della sola ricerca testuale per nome; manca un filtraggio strutturato (per fascia di prezzo, per disponibilità) e l'ordinamento direzionale dei risultati; mancano inoltre gli indici espliciti sulle chiavi esterne
Motivazione	All'aumentare dell'assortimento, l'assenza di filtraggio per fascia di prezzo/disponibilità e di ordinamento direzionale rende difficoltosa l'individuazione dei prodotti. Si tratta di una funzionalità standard per un sistema di e-commerce, che ne migliora significativamente l'usabilità; trattandosi dell'aggiunta di una nuova funzionalità, la modifica è di natura perfettiva (evolutiva)
Obiettivo	Introdurre il filtraggio del catalogo per fascia di prezzo (minimo-massimo) e disponibilità (soli prodotti in giacenza), con ordinamento per prezzo/nome in direzione crescente/decrecente, e aggiungere gli indici di supporto. Il tutto preservando i flussi esistenti: visualizzazione del catalogo, ordinamento semplice, dettaglio prodotto e ricerca testuale per nome
Analisi di Fattibilità (Feasibility)	Intervento confinato all'area catalogo e a basso rischio sui flussi esistenti, reso fattibile da: il livello di servizio del catalogo già disponibile, su cui la funzionalità si innesta in modo pulito; l'assenza di nuove dipendenze esterne; la separazione del nuovo percorso di ricerca

	filtrata da quello di ordinamento semplice (preservato), con instradamento alla nuova operazione solo in presenza di criteri di filtro o di direzione
Analisi dei Rischi (Risk)	Rischio sul ripple dei dati seed: l'arricchimento del catalogo, necessario per rendere verificabili i filtri, può alterare gli oracoli dei casi di visualizzazione esistenti. Mitigazione: verifica e riallineamento mirato di tali oracoli. Rischio di regressione sull'ordinamento esistente: mitigato mantenendo intatta l'operazione di ordinamento semplice e separando il nuovo percorso. Rischio di SQL injection sul nuovo parametro di direzione: mitigato ammettendo colonna e direzione solo via lista bianca (coerentemente con la messa in sicurezza), con i valori di prezzo sempre come parametri di query. Rischio di schema/seed: la modifica di indici e dati seed richiede la ricreazione della base di dati come preconditione della regressione, con la fixture di ripristino mantenuta in sincronia
Effetto atteso sul testing	A differenza delle Change Request correttiva e preventiva, che fanno flippare l'esito dei casi a oracolo invariato, CR_04 è evolutiva e introduce un delta della specifica dei test: un nuovo requisito di filtraggio del catalogo e una nuova famiglia di casi di test (Category Partition + analisi ai valori limite sulla fascia di prezzo; verifica della disponibilità; direzione dell'ordinamento; criterio non ammesso → pagina d'errore, a conferma dell'anti-SQL-injection). Sui requisiti esistenti del catalogo si attende invarianza, salvo il riallineamento degli oracoli dovuto all'arricchimento del seed

Analisi e opzioni d'implementazione (decisioni di scope):

- Perimetro dei filtri: fascia di prezzo (minimo-massimo), disponibilità (soli prodotti in giacenza), ordinamento per prezzo/nome con direzione crescente/decrescente. Esclusione del filtro per tipo: i dati attuali del catalogo presentano un unico valore di tipo → un filtro per categoria non sarebbe né significativo né verificabile; il criterio è comunque predisposto a una futura estensione.
- Architettura: introduzione di una nuova operazione di ricerca filtrata nel livello dati e nel livello di servizio, con condizioni parametriche sulla query; l'operazione è distinta sia dalla visualizzazione/ordinamento semplice (preservata) sia dalla ricerca testuale per nome (preservata). I criteri di filtro sono raccolti in un oggetto dedicato.
- Anti-SQL-injection preservato: i criteri di ordinamento (colonna e direzione) sono ammessi solo se presenti in una lista bianca; i valori di prezzo transitano sempre come parametri della query. Il criterio incoerente (prezzo minimo maggiore del massimo) è reso come risultato vuoto, non come errore di sistema (la pagina d'errore resta riservata al criterio di ordinamento non ammesso).
- Indici: aggiunta di indici di supporto al filtraggio (sul prezzo e sulla disponibilità) e formalizzazione esplicita degli indici sulle chiavi esterne già previsti in fase di proposta. Questi ultimi sono già creati automaticamente dal motore transazionale → l'intervento è di natura documentale/di robustezza, non prestazionale-critico.
- Arricchimento dei dati seed: introduzione di prodotti che abilitano la verifica dei filtri (almeno un prodotto non in giacenza per la disponibilità; una dispersione di prezzo per

l'analisi ai valori limite). La fixture di ripristino del seed è aggiornata in parallelo con gli stessi dati.

3.4.2 Impact analysis

3.4.2.1 Identificazione dello Start Impact Set (SIS)

#	Artefatto	Descrizione dell'intervento	Grado
1	ProductFilter.java (<i>nuovo</i>)	Oggetto dei criteri di filtro (prezzo minimo/massimo, sola disponibilità, colonna e direzione di ordinamento)	Forte
2	ProductModel.java (<i>interfaccia</i>)	Nuova operazione di ricerca filtrata del catalogo	Forte
3	ProductModelDM.java	Implementazione della ricerca filtrata: condizioni parametriche su prezzo/disponibilità; ordinamento per colonna e direzione su lista bianca; mappatura completa del bean (inclusi tipo e colore)	Forte
4	ProductService.java	Operazione di filtraggio nel livello di servizio e normalizzazione/validazione dei criteri	Medio
5	ProductControl.java	Lettura dei parametri di filtro/direzione e instradamento alla ricerca filtrata (ordinamento semplice preservato)	Medio
6	ProductView.jsp	Modulo di filtro e ordinamento nella vista del catalogo	Medio
7	ProductViewAdmin.jsp	Modulo di filtro e ordinamento nella vista amministrativa del catalogo	Medio
8	db/storage.sql	Indici (prezzo, disponibilità, chiavi esterne) e arricchimento dei dati seed del catalogo	Forte

|SIS| = 8. Un artefatto di verifica è stato modificato in parallelo (escluso dalle metriche IR): la fixture di ripristino del seed, arricchita in sincronia con lo schema.

3.4.2.2 Identificazione del Candidate Impact Set (CIS)

#	Artefatto (oltre al SIS)	Perché impattato (ripple)	Grado
9	ProductBean.java	popolato dalla ricerca filtrata (valorizzazione di tipo e colore): dipendenza in lettura/scrittura, nessuna modifica strutturale attesa	Debole

10	error-page.jsp	destinazione del reindirizzamento in caso di criterio di ordinamento non ammesso (riuso del ramo d'errore già esistente)	Debole
----	----------------	--	--------

|CIS| = 10 (8 del SIS + 2 di ripple). Esclusi dal perimetro, come per le CR precedenti: la matrice di tracciabilità (da aggiornare con il nuovo requisito e la nuova famiglia di casi di test) e i test case stessi. Non incidono sulle metriche IR ma vanno completati prima della chiusura della Change Request.

3.4.3 Risultati dell'impact analysis

Confronto tra il Candidate Impact Set predetto e l'Actual Impact Set reale, sui soli artefatti di prodotto.

3.4.3.1 Identificazione del Discovered Impact Set (DIS)

|DIS| = 0. La funzionalità evolutiva si è innestata esattamente sul perimetro previsto (livello dati/servizio/controllo/vista + schema), senza impatti emersi non pianificati.

3.4.3.2 Identificazione del False Positive Impact Set (FPIS)

#	Artefatto (nel CIS)	Perché non modificato
1	ProductBean.java	La valorizzazione di tipo e colore da parte della ricerca filtrata utilizza i metodi di scrittura già esistenti del bean → nessuna modifica strutturale
2	error-page.jsp	La pagina è la destinazione riusata del ramo d'errore: il reindirizzamento è impostato nel controllo, la pagina resta identica

|FPIS| = 2. Over-estimation di natura conservativa: due dipendenze previste (una in scrittura tramite interfaccia già presente, una in sola destinazione di reindirizzamento) non hanno richiesto modifica dell'artefatto.

3.4.3.3 Identificazione dell'Actual Impact Set (AIS)

Componenti realmente modificate = $(CIS - FPIS) \cup DIS = CIS - FPIS$ (poiché $DIS = \emptyset$).

#	Artefatto	Modifica effettiva
1	ProductFilter.java (<i>nuovo</i>)	Oggetto dei criteri di filtro (prezzo minimo/massimo, sola disponibilità, colonna e direzione di ordinamento)
2	ProductModel.java (<i>interfaccia</i>)	Firma dell'operazione di ricerca filtrata del catalogo
3	ProductModelDM.java	Ricerca filtrata: condizioni parametriche su prezzo/disponibilità, ordinamento per colonna e direzione su lista bianca, mappatura completa del bean

4	ProductService.java	Operazione di filtraggio e normalizzazione dei criteri (prezzi negativi ignorati; intervallo con minimo maggiore del massimo → risultato vuoto)
5	ProductControl.java	Lettura dei parametri di filtro/direzione e instradamento alla ricerca filtrata; ordinamento semplice preservato; riuso del ramo d'errore
6	ProductView.jsp	Modulo di filtro e ordinamento nella vista del catalogo
7	ProductViewAdmin.jsp	Modulo di filtro e ordinamento nella vista amministrativa
8	db/storage.sql	Indici su prezzo, disponibilità e chiavi esterne; arricchimento dei dati seed (prodotto non in giacenza e valori di prezzo ai bordi del dominio)

|AIS| = 8 (di prodotto). Coincide con il SIS previsto: la predizione a priori è risultata completa (nessun impatto emerso) e leggermente sovrastimata (due dipendenze non tradotte in modifica).

3.4.4 Stato di verifica (regressione)

La regressione test-all è stata eseguita sulla suite estesa con la nuova famiglia di casi, con fixture di ripristino del seed attiva prima di ciascun caso. Esito complessivo: 118 PASS / 10 FAIL su 128 casi.

Oggetto	Esito
Nuovo requisito (filtraggio del catalogo)	Eseguito. I casi di filtro/ordinamento a risultato ridotto sono superati; alcuni casi a risultato ampio risultano falliti per limite dell'oracolo (la verifica testuale via interfaccia non rileva gli elementi oltre la finestra visibile del componente grafico del catalogo): il comportamento del prodotto è verificato corretto. Tali fallimenti sono classificati come incident di strumentazione di test, non come difetti di prodotto
Caso di bordo (prezzo massimo al minimo del dominio)	Fallito: risultato vuoto al bordo. Registrato come finding da diagnosticare (precisione in virgola mobile del confronto, da verificare rispetto a un possibile difetto del filtro)
Invarianza sui requisiti esistenti (visualizzazione, ordinamento semplice, dettaglio, ricerca per nome)	Preservata, nessuna regressione funzionale sui flussi esistenti del catalogo

L'esito è registrato nel registro di regressione e confluisce nei report finali di esecuzione e di incident, insieme al delta della specifica dei test (nuovo requisito e nuova famiglia di casi).

Coerentemente con il principio di integrità della verifica, gli esiti reali sono documentati e classificati, senza modifica a posteriori degli oracoli di test.

3.4.5 Metriche

Per CR_04 ($|CIS| = 10$, $|AIS| = 8$, $|CIS \cap AIS| = 8$, $|DIS| = 0$, $|FPIS| = 2$):

Recall = 1.00, Precision = 0.80, F-measure = 0.89, EstError = 0.20.

Il Recall massimo conferma piena adeguatezza, nonostante si tratti della prima CR evolutiva del progetto. La Precision è la più alta fra le cinque CR: solo 2 falsi positivi su 10, entrambi dipendenze non tradotte in modifica strutturale. F-measure ed EstError sono rispettivamente il valore più alto e più basso del progetto, coerenti con un perimetro ben delimitato, innestato su un'architettura a livelli già consolidata dal re-engineering.

3.5 CR_05

3.5.1 Identificazione, analisi e valutazione

Campo	Valore
Identificativo	CR_05
Titolo	Containerizzazione del sistema con Docker
Tipologia	Adattiva
Priorità	Bassa
Dipendenze	CR_01 e CR_02 (sistema corretto e re-ingegnerizzato: sorgente dati unico su DataSource)
Origine	Esigenza di adattare il sistema a un ambiente di deployment riproducibile con un singolo comando di orchestrazione, eliminando la dipendenza dall'ambiente locale dello sviluppatore
Effetto atteso sul testing	Nessun delta della specifica dei test: la CR è infrastrutturale e non modifica requisiti né spazio d'input. La verifica è una verifica di portabilità (il comportamento osservabile via interfaccia deve restare identico al deployment di sviluppo)

Disallineamento a priori ↔ stato reale (rilevante per i risultati). L'analisi preliminare era stata redatta sullo stato as-delivered: prevedeva la lettura delle credenziali da variabile d'ambiente nel pool di connessioni custom. Tale componente è stato eliminato dalla manutenzione perfetta in favore di un DataSource unico gestito dal container; l'intervento previsto decade

e la parametrizzazione è ricollocata sulla configurazione del sorgente dati. Questo è l'esito atteso di una Change Request adattiva che chiude la sequenza dopo il re-engineering.

3.5.2 Impact analysis a priori (SIS → CIS)

Start Impact Set (SIS) — artefatti di prodotto creati direttamente (tutti nuovi): build Maven del pacchetto applicativo, immagine Docker multi-stage, configurazione del sorgente dati parametrizzata per il container, file di orchestrazione applicazione + base di dati, file di esclusione del contesto di build, e la guida di avvio per l'esecuzione in Docker (opzionale).

|SIS| = 6.

Candidate Impact Set (CIS) = SIS + componenti raggiunte per ripple (tutte a grado Debole): la configurazione del sorgente dati di sviluppo (in parità con la variante d'immagine), lo script di inizializzazione della base di dati (atteso in sola lettura), le quattro librerie applicative (ora gestite dalla build Maven), le due guide d'ambiente per l'esecuzione locale, e il pool di connessioni custom (destinatario teorico dell'intervento previsto a priori). Controller, servizi, modello, viste e specifica dei test non sono impattati. **|CIS| = 15 (6 + 9).**

3.5.3 Risultati dell'impact analysis

3.5.3.1 Identificazione del Discovered Impact Set (DIS)

|DIS| = 0. Nessun artefatto di prodotto è stato modificato al di fuori del CIS previsto: il footprint è interamente costituito da nuovi file d'infrastruttura, come atteso per una CR adattiva a comportamento preservato.

3.5.3.2 Identificazione del False Positive Impact Set (FPIS)

#	Artefatto (nel CIS)	Perché non modificato
1	Pool di connessioni custom	Non esiste più (eliminato dal re-engineering): l'intervento di lettura da ambiente previsto a priori è inapplicabile
2	Configurazione del sorgente dati di sviluppo	Lasciata invariata; l'immagine usa la variante parametrizzata dedicata
3-6	Librerie applicative (4 jar)	Non alterate: solo prese in carico dalla build Maven
7	Guida d'ambiente (configurazione IDE)	Guida di sviluppo affiancata dal flusso Docker, non modificata
8	Guida d'ambiente (esecuzione locale)	Idem
9	Guida di avvio in Docker	Prevista come opzionale, non redatta in questa fase

|FPIS| = 9. Over-estimation strutturale, tipica di una CR adattiva a valle del re-engineering: un intervento decaduto (componente inesistente), dipendenze in sola lettura/parità e artefatti documentali affiancati.

3.5.3.3 Identificazione dell'Actual Impact Set (AIS)

#	Artefatto	Modifica effettiva
1	Build Maven (<i>nuovo</i>)	Build del pacchetto applicativo per l'immagine
2	Dockerfile (<i>nuovo</i>)	Immagine multi-stage (build → runtime), driver nella libreria del container
3	Configurazione del sorgente dati per il container (<i>nuova</i>)	Sorgente dati parametrizzato via variabili d'ambiente
4	File di orchestrazione (<i>nuovo</i>)	Applicazione + base di dati, con healthcheck e inizializzazione automatica
5	File di esclusione del contesto di build (<i>nuovo</i>)	Riduzione del contesto di build
6	db/storage.sql	Riallineamento della nomenclatura dei dati seed per la compatibilità con l'ambiente di esecuzione (previsto in sola lettura, risultato modificato)

|AIS| = 6. Il perimetro reale coincide con i cinque nuovi file d'infrastruttura più il solo adeguamento dello script di inizializzazione; nessuna modifica al codice applicativo → comportamento preservato per costruzione.

3.5.4 Stato di verifica (portabilità)

La verifica è una regressione di portabilità: l'applicazione containerizzata deve comportarsi in modo identico al deployment di sviluppo. L'allineamento del contesto applicativo dell'immagine a quello di sviluppo ha consentito di eseguire la suite di sistema completa contro il container senza modificarla.

La regressione test-all è stata eseguita contro il container (128 casi): esito 118 PASS / 10 FAIL, coincidente con quello del deployment di sviluppo → nessuna regressione di portabilità. I 10 fallimenti sono pre-esistenti e indipendenti dall'ambiente (limiti dell'oracolo su un componente grafico del catalogo, un caso di bordo del filtro, un oracolo di conteggio da riallineare, un incident di prodotto residuo sulla validazione del codice fiscale), classificati e non imputabili alla containerizzazione.

Un successivo intervento conservativo ha irrobustito l'ambiente containerizzato, esternalizzazione dei segreti, esecuzione del runtime con utente non privilegiato, healthcheck

dell'applicazione e politica di riavvio, parità del contesto applicativo, senza alcun impatto sul comportamento applicativo.

3.5.5 Metriche

Per CR_05 ($|CIS| = 15$, $|AIS| = 6$, $|CIS \cap AIS| = 6$, $|DIS| = 0$, $|FPIS| = 9$).

Il successivo intervento di irrobustimento resta escluso, essendo conservativo e non parte della containerizzazione valutata):

Recall = 1.00, Precision = 0.40, F-measure = 0.57, EstError = 0.60.

Il Recall massimo conferma ancora piena adeguatezza, malgrado la natura infrastrutturale della CR. La Precision è la più bassa del progetto, per ragioni strutturali non patologiche: il componente bersaglio teorico dell'analisi a priori (il pool di connessioni custom) non esiste più perché eliminato dal re-engineering, e altre componenti previste sono dipendenze in sola lettura o documentazione affiancata. L'errore di stima, il più alto del progetto, resta interamente da falsi positivi: nessun impatto ad alto costo è stato mancato, bilanciamento coerente con una CR adattiva a basso rischio, il cui obiettivo primario è non perdere impatti, non minimizzare le sovrastime.

3.6 Metriche: sintesi complessiva

Change Request	Recall	Precision	F-measure	EstError
CR_01	0.95	0.69	0.80	0.35
CR_02	1.00	0.79	0.88	0.21
CR_03	1.00	0.60	0.75	0.40
CR_04	1.00	0.80	0.89	0.20
CR_05	1.00	0.40	0.57	0.60

In quattro Change Request su cinque il Recall è massimo: l'unico impatto realmente mancato in tutto il progetto è quello di CR_01, di entità minima. La Precision varia invece sensibilmente, ed è la dimensione su cui si misura la reale differenza di qualità tra le stime: più alta dove il perimetro era ben delimitato e innestato su un'architettura già consolidata (CR_04, CR_02), più bassa dove l'analisi a priori scontava scelte di scope riviste in corso d'opera o componenti bersaglio nel frattempo eliminati da manutenzioni precedenti (CR_03, CR_05). In tutti i casi l'errore di stima è dominato dai falsi positivi (sovrastime conservative a basso costo) piuttosto che da impatti scoperti tardivamente: la disciplina di non riscrivere il Candidate Impact Set a

posteriori, mantenuta in modo uniforme lungo le cinque CR, rende queste metriche una misura onesta della capacità predittiva dell'analisi, non un adattamento retrospettivo del risultato.

4. Sistema modificato

Le cinque Change Request pianificate sono state completate, nell'ordine d'esecuzione CR_01 → CR_02 || CR_03 → CR_04 → CR_05, ciascuna chiusa e taggata, con una successiva patch conservativa di irrobustimento a valle di CR_05, coprendo tutte le tipologie di manutenzione previste da ISO/IEC 14764: correttiva (CR_01), perfettiva (CR_02, re-engineering; CR_04, nuova funzionalità), preventiva (CR_03) e adattiva (CR_05). Questo paragrafo descrive lo stato consegnabile del sistema, riprendendo la struttura della panoramica del sistema as-delivered con i contenuti aggiornati allo stato post-manutenzione; il dettaglio di ciascun intervento resta nelle sezioni 3.1-3.5.

4.1 Descrizione del sistema

Vaporant resta, nella sua identità, la piattaforma descritta nella panoramica del sistema as-delivered: un e-commerce di sigarette elettroniche ed accessori da svapo, con consultazione del catalogo, gestione del carrello, registrazione/autenticazione, gestione del profilo, acquisto con generazione della fattura in PDF e un'area amministrativa per la gestione dei prodotti. Nessuna Change Request ha alterato questa identità funzionale di fondo: le cinque CR hanno corretto il comportamento non conforme ai requisiti (CR_01), ristrutturato l'architettura interna a comportamento preservato (CR_02), messo in sicurezza il sistema (CR_03), arricchito il catalogo con filtraggio/ordinamento avanzato (CR_04) e adattato il sistema a un ambiente di deployment containerizzato (CR_05).

Rispetto allo stato as-delivered, la piattaforma consegnata presenta due differenze funzionali/di distribuzione osservabili dall'utente:

- **Filtraggio e ordinamento avanzato del catalogo** (CR_04): oltre alla consultazione e all'ordinamento semplice già presenti, il catalogo può ora essere filtrato per fascia di prezzo (minimo-massimo) e per disponibilità (solì prodotti in giacenza), con ordinamento per prezzo o nome in direzione crescente/decescente. La ricerca testuale per nome, il dettaglio prodotto e l'ordinamento semplice restano invariati.
- **Distribuzione containerizzata** (CR_05): il sistema è distribuito come immagine Docker (Tomcat 9, build Maven multi-stage) orchestrata con un file di composizione che avvia applicazione e base di dati con inizializzazione automatica dello schema; l'ambiente si avvia con un solo comando. Il deployment as-delivered (ambiente locale di sviluppo) resta comunque disponibile come modalità di sviluppo.

Tutte le altre funzionalità restano presenti e comportamentalmente conformi ai requisiti, a differenza dello stato as-delivered, dove diversi flussi (login, registrazione, checkout, inserimento prodotto) erano bloccati da difetti (sezione 3.1 CR_01).

4.2 Attori del sistema

Nessuna delle cinque Change Request ha introdotto, rimosso o modificato le responsabilità di un attore: gli attori del sistema restano invariati rispetto allo stato as-delivered. Il vincolo di dominio resta tipo IN ('user','admin'), con generalizzazione totale e disgiunta:

- **Ospite** (utente non autenticato): consultazione del catalogo (ora con filtraggio/ordinamento avanzato, CR_04), ricerca prodotti, gestione del carrello, registrazione, consultazione delle pagine informative.
- **Cliente** (utente registrato autenticato): tutte le funzionalità dell'ospite, gestione del profilo e degli indirizzi, acquisto con checkout, consultazione dello storico ordini, download della fattura.
- **Amministratore** (utente con privilegi di gestione del catalogo): le funzionalità del cliente più l'inserimento e la rimozione di prodotti tramite la vista amministrativa dedicata.

L'invarianza degli attori è un dato di fatto rilevante, non un'omissione di analisi: le cinque CR intervengono tutte su comportamento, architettura interna, sicurezza o ambiente di distribuzione, mai sul modello degli attori né sulle operazioni ad essi consentite.

4.3 Requisiti funzionali

La quasi totalità dei 25 requisiti funzionali ricostruiti in sede di reverse engineering resta invariata nel testo del requisito: secondo la regola dell'oracolo, gli interventi di CR_01 (correttiva) e di CR_03 (preventiva) hanno corretto il comportamento del sistema non conforme al requisito, facendo flippare l'esito dei casi di test da Failed a Passed, senza modificare la formulazione dei requisiti stessi. L'unico requisito nuovo, introdotto da CR_04, è RF_GC_26, *Filtrare il catalogo*.

Categoria	ID	Breve descrizione	Priorità	Stato post-manutenzione
Gestione Autenticazione	RF_GA_1	Login	Alta	Invariato (comportamento corretto da CR_01)
Gestione Autenticazione	RF_GA_2	Logout	Alta	Invariato

Gestione Registrazione	RF_GR_3	Registrare cliente	Alta	Invariato (comportamento corretto da CR_01)
Gestione Profilo	RF_GP_4	Visualizzare profilo	Media	Invariato
Gestione Profilo	RF_GP_5	Modificare e-mail	Media	Invariato (validazione server aggiunta da CR_02)
Gestione Profilo	RF_GP_6	Modificare numero di telefono	Media	Invariato (validazione server aggiunta da CR_02)
Gestione Profilo	RF_GP_7	Modificare password	Media	Invariato (cifatura da CR_03; sincronizzazione di sessione da CR_02)
Gestione Profilo	RF_GP_8	Visualizzare indirizzi	Media	Invariato
Gestione Profilo	RF_GP_9	Aggiungere indirizzo	Media	Invariato (validazione server aggiunta da CR_02)
Gestione Profilo	RF_GP_10	Visualizzare storico ordini	Media	Invariato
Gestione Catalogo	RF_GC_11	Visualizzare catalogo	Alta	Invariato (dati seed ampliati da CR_04)
Gestione Catalogo	RF_GC_12	Ordinare catalogo	Media	Invariato (whitelist anti-SQL-injection aggiunta da CR_03)
Gestione Catalogo	RF_GC_13	Visualizzare dettaglio prodotto	Alta	Invariato (robustezza su identificativo non valido aggiunta da CR_02)
Gestione Catalogo	RF_GC_14	Ricerca prodotto	Media	Invariato
Gestione Catalogo	RF_GC_26	Filtrare il catalogo	Media	Nuovo (CR_04)

Gestione Carrello	RF_GCR_15	Visualizzare carrello	Alta	Invariato
Gestione Carrello	RF_GCR_16	Aggiungere prodotto al carrello	Alta	Invariato (guardia su prodotto inesistente da CR_02)
Gestione Carrello	RF_GCR_17	Rimuovere prodotto dal carrello	Media	Invariato
Gestione Carrello	RF_GCR_18	Aggiornare quantità prodotto nel carrello	Media	Invariato (validazione server della quantità da CR_02)
Gestione Ordine	RF_GO_19	Visualizzare checkout	Alta	Invariato (protezione via filtro di autenticazione da CR_02)
Gestione Ordine	RF_GO_20	Effettuare acquisto	Alta	Invariato (comportamento corretto da CR_01; atomicità della transazione da CR_02)
Gestione Ordine	RF_GO_21	Visualizzare riepilogo dell'ordine effettuato	Media	Invariato
Gestione Ordine	RF_GO_22	Scaricare fattura	Media	Invariato (comportamento corretto da CR_01: generazione via codice)
Gestione Prodotti	RF_GPR_23	Inserire prodotto	Alta	Invariato (comportamento corretto da CR_01: campi tipo/colore)
Gestione Prodotti	RF_GPR_24	Rimuovere prodotto	Alta	Invariato (comportamento corretto da CR_01 sulla vista; controllo di ruolo sulla servlet aggiunto da CR_03)
Gestione Generale	RF_GG_25	Consultare pagine informative	Bassa	Invariato

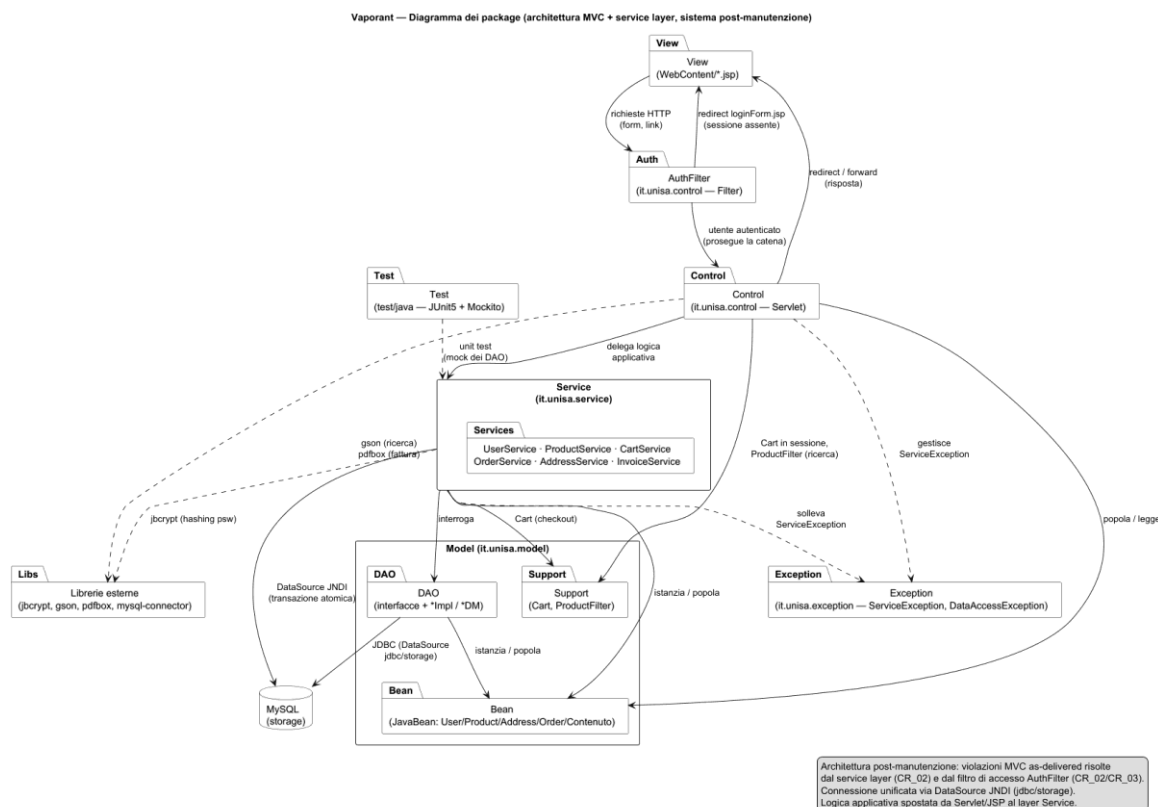
Il dettaglio testuale dei 25 requisiti originali resta quello della descrizione as-delivered. Il requisito nuovo è definito come segue:

RF_GC_26, Filtrare il catalogo (Gestione Catalogo, priorità Media): il sistema permette a ospite/cliente/amministratore di filtrare il catalogo per fascia di prezzo (minimo-massimo) e disponibilità (soli prodotti in giacenza), e di ordinarne i risultati per prezzo o nome in direzione crescente o decrescente. Il dettaglio completo dei casi di test derivati (Category Partition, analisi ai valori limite) è nella specifica di test aggiornata.

La colonna "Stato post-manutenzione" richiama, per ciascun requisito toccato da una correzione comportamentale, la Change Request di riferimento; il dettaglio degli interventi è nelle sezioni 3.1-3.5.

4.4 Architettura del sistema

Il cambiamento architetturale più importante del progetto è quello introdotto da CR_02 (sezione 3.2): il sistema non è più una Model-View-Controller con logica applicativa dispersa nelle Servlet e nelle JSP, ma un'architettura MVC con livello di servizio, con ulteriori interventi di sicurezza (CR_03) e di containerizzazione (CR_05) sovrapposti a valle.



- **Livello di servizio.** Un nuovo package dedicato ospita la logica applicativa, in precedenza inglobata nelle Servlet: UserService (autenticazione, registrazione, gestione del profilo, cifratura/verifica della password), OrderService (checkout, atomicità della transazione d'acquisto, lettura degli ordini per utente), CartService (gestione del

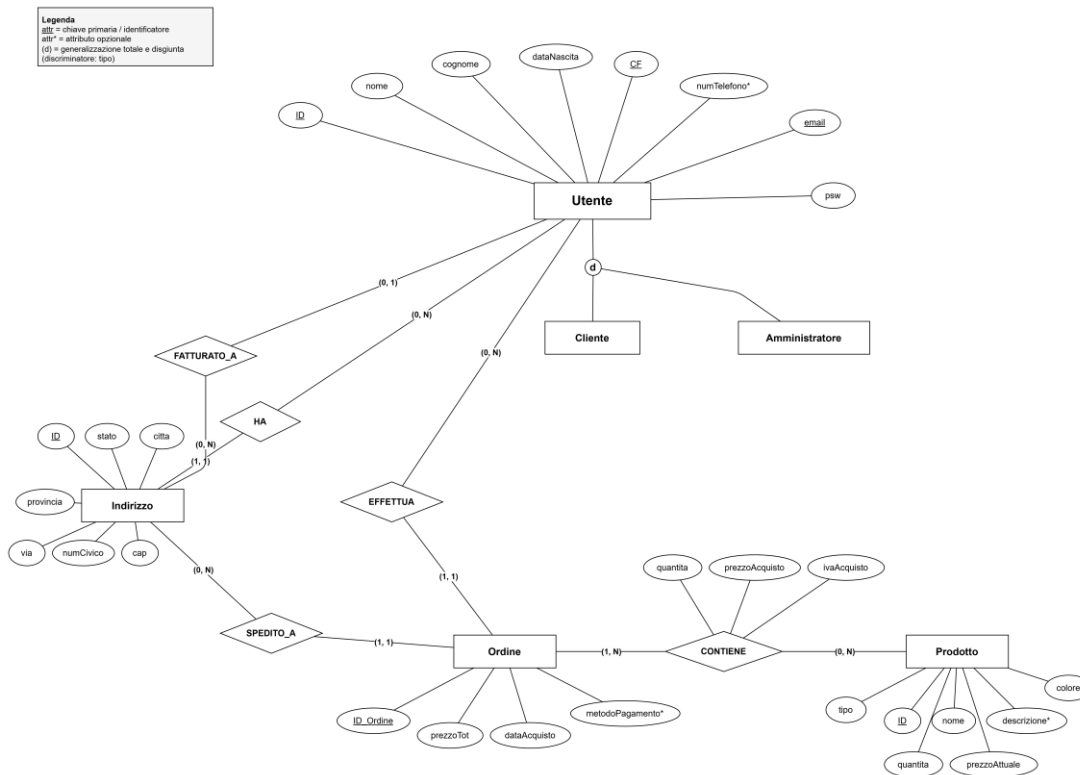
carrello), ProductService (catalogo, ricerca per nome, filtraggio/ordinamento avanzato introdotto da CR_04), InvoiceService (centralizzazione del calcolo dell'IVA) e AddressService (gestione e validazione degli indirizzi). Le Servlet sono state ricablate su questi servizi, riducendosi a un ruolo di puro controllo.

- **Gerarchia di eccezioni applicative.** Un nuovo package introduce DataAccessException (errori del livello dati) e ServiceException (errori del livello di servizio), sostituendo la gestione precedente basata perlopiù sulla stampa diretta dell'errore.
- **Autenticazione e controllo degli accessi.** Un filtro di autenticazione, di nuova introduzione, protegge le risorse dell'area riservata cliente: profilo, carrello/checkout, modulo e gestione degli indirizzi, modifica del profilo, ordine, fattura. La risorsa di gestione prodotti (area amministrativa) è deliberatamente esclusa dal filtro: il controllo di ruolo su tale risorsa era già stato corretto sulla vista da CR_01 ed è stato poi completato a livello di servlet da CR_03, per non duplicare l'intervento fra le due Change Request.
- **Connessioni e transazioni.** Le connessioni al database sono state unificate su un solo DataSource, eliminando il pool di connessioni custom responsabile dell'incident di isolamento/transazione e della strategia di connessione eterogenea del sistema as-delivered. I confini delle transazioni sono ora nel livello di servizio, che apre la connessione e governa commit/rollback (ad esempio l'atomicità del checkout), passando la connessione ai DAO.
- **Uniformazione del livello dati.** I DAO sono stati uniformati per naming dei metodi e per contratto di ritorno (restituzione di null su identificativo inesistente anziché un bean di default; liste vuote anziché null sulle ricerche multiple), correggendo le incoerenze rilevate nel reverse engineering.
- **Interventi di sicurezza (CR_03).** A questa architettura si sovrappongono: cifratura BCrypt delle password, in sostituzione della memorizzazione in chiaro; whitelist anti-SQL-injection sul parametro di ordinamento del catalogo, che correggeva la concatenazione diretta del parametro nella query; controllo del ruolo amministrativo sulla rimozione prodotto anche a livello di servlet, a completamento del controllo già corretto sulla vista da CR_01; mascheramento della password nella rappresentazione testuale dell'utente.
- **Containerizzazione (CR_05).** Il sistema è distribuito come immagine Docker costruita con un file di build multi-stage (build Maven del pacchetto applicativo, seguita da un'immagine runtime Tomcat 9), orchestrata insieme al servizio di base di dati. Il sorgente dati è parametrizzato via variabili d'ambiente, sostituendo la configurazione cablata dell'ambiente di sviluppo.

4.5 Database

Lo schema del database ha subito due modifiche strutturali reali rispetto allo stato as-delivered, entrambe introdotte nell'ambito di CR_02 e CR_03 (dettaglio nelle sezioni 3.2 e 3.3).

4.5.1 Modello EER



4.6 Stato di verifica finale

Durante l'intero progetto, ogni modifica (Change Request - CR) è stata testata: i test non sono mai stati alterati per forzare un risultato.

Ecco il riepilogo dei risultati per ogni fase:

- **CR_01 (Correzioni funzionali):** Il sistema ha superato 80 test su 113. I 33 test falliti erano difetti reali già previsti: alcuni sono stati risolti in questa fase, altri programmati per le modifiche successive.
- **CR_02 (Ristrutturazione dell'architettura):** Il codice è stato migliorato mantenendo le stesse funzionalità. I test superati sono saliti a 110 su 113 (il conteggio include la correzione di un "falso allarme" dovuto a un errore nel test stesso, non nel sistema).
- **CR_03 (Sicurezza):** Sono state aggiunte protezioni (cifatura, controlli anti-intrusione e permessi). Il sistema è diventato più stabile, raggiungendo 112 test superati e 1 solo fallito.

- **CR_04 (Nuove funzionalità):** È stato aggiunto un sistema di filtri e ordinamento per il catalogo. Questo ha richiesto l'aggiunta di nuovi casi di test, portando il totale da 113 a 128.
- **CR_05 (Containerizzazione):** Il sistema è stato "pacchettizzato" per garantirne la portabilità. I test hanno confermato che il container funziona esattamente come l'ambiente di sviluppo, senza introdurre nuovi errori.

L'ultima sessione di test (che copre sia la CR_04 che la CR_05) si è conclusa con **118 test superati e 10 falliti**.

Questi 10 fallimenti non sono gravi e sono così classificati da noi:

- **9 falsi negativi (errori nei test):** Il sistema funziona correttamente, ma i test falliscono per problemi tecnici (questioni di interfaccia grafica, arrotondamenti di calcolo e dati di partenza non aggiornati). I test verranno sistemati in seguito.
- **1 difetto reale del prodotto:** Il sistema accetta erroneamente un Codice Fiscale composto da soli spazi vuoti. Essendo un problema minore e fuori dal perimetro di queste 5 CR, verrà risolto a parte con un intervento dedicato.